



Dokumentacja projektu *KeyForge*

Podstawy konteneryzacji i architektury mikroserwisów
Politechnika Łódzka

Paweł Pater 246354
Jakub Klimczyk 246327

Spis treści

Wstęp	2
Opis Systemu	2
Cele projektu	2
Role aplikacyjne	3
Założenia techniczne	3
Kluczowe Funkcjonalności	3
Diagramy Systemowe	4
Diagram przypadków użycia (UML)	4
Diagram Sekwencji	5
Diagram Związków Encji (ERD)	6
Diagram Przejść Stanów (STD)	9
Implementacja Warstwy Danych	10
Mapowanie obiektowo-relacyjne (ORM)	10
Tryb połączenia z bazą danych	10
Dokumentacja interfejsów API (Swagger/OpenAPI)	12
Weryfikacja i testowanie interfejsów API	12
Interfejs Swagger (OpenAPI)	12
Testy w środowisku Insomnia	15
Architektura Konteneryzacji	17
Zasady Mapowania Portów	17
Konfiguracja Docker Compose	17
Skonteneryzowane Usługi	20
Sieć i Komunikacja	20
Konfiguracja i Uruchomienie Systemu	21
Wymagania systemowe	21
Konfiguracja zmiennych środowiskowych	21
Procedura uruchomienia	21
Punkty dostępowe	22
Interfejs użytkownika	23
Panel klienta	23
Panel administratora	35
Autoryzacja i obsługa błędów dostępu	41
Podsumowanie	44
Wnioski dotyczące architektury Backendowej	44
Wnioski dotyczące Interfejsu Użytkownika	45

Wstęp

Niniejsza dokumentacja stanowi opis techniczny systemu **KeyForge** — nowoczesnej platformy e-commerce dedykowanej dystrybucji kluczy do gier cyfrowych. Celem dokumentu jest przedstawienie architektury systemu, szczegółowy opis poszczególnych modułów oraz specyfikacja środowiska kontenerowego, w którym aplikacja została osadzona.

Dokumentacja zawiera:

- Opis funkcjonalny i biznesowy systemu.
- Specyfikację architektury mikroserwisowej.
- Szczegółową listę skonteneryzowanych usług wraz z ich rolami.
- Schematy przepływu danych i struktury bazy danych.
- Konfigurację Docker Compose zarządzającą całym ekosystemem.
- Szczegółowy opis i wygląd interfejsu użytkownika.

Opis Systemu

KeyForge to aplikacja webowa inspirowana platformami takimi jak Eneba czy G2A, umożliwiająca użytkownikom przeglądanie, zakup oraz ocenianie gier cyfrowych. System został zaprojektowany w architekturze mikroserwisowej przy użyciu technologii **Spring Boot** dla warstwy backendowej oraz **React.js** dla warstwy frontendowej.

Cele projektu

Głównym celem projektu jest stworzenie skalowalnego ekosystemu aplikacyjnego, który umożliwi:

- Automatyzację procesu sprzedaży i natychmiastowe wydawanie zakupionych produktów
- Implementację zaawansowanego systemu lojalnościowego „KeyPoints”, zwiększającego retencję użytkowników
- Wykorzystanie sztucznej inteligencji do podniesienia jakości treści generowanych przez użytkowników (moderacja i streszczenia opinii)
- Zapewnienie wysokiego poziomu bezpieczeństwa transakcji oraz ochrony danych osobowych poprzez integrację z zewnętrznymi systemami autoryzacji (Keycloak) i płatności (Stripe)

Role aplikacyjne

W systemie zdefiniowano dwie główne role użytkowników, z których każda posiada odrębny zakres uprawnień i dostęp do dedykowanych modułów funkcjonalnych:

1. Użytkownik (Klient):

- Przeglądanie katalogu produktów z wykorzystaniem dynamicznego filtrowania.
- Zarządzanie koszykiem zakupowym i realizacja płatności elektronicznych.
- Dostęp do panelu profilu, historii zamówień oraz odbiór zakupionych kluczy cyfrowych.
- Udział w programie lojalnościowym i zbieranie punktów KeyPoints.
- Wystawianie ocen i opinii o zakupionych produktach.

2. Administrator:

- Zarządzanie asortymentem: dodawanie, edycja (CRUD) oraz usuwanie produktów z bazy PostgreSQL.
- Nadzór nad procesami sprzedażowymi i wgląd w szczegółowe dane transakcyjne (np. Payment ID).
- Moderacja opinii użytkowników przy wsparciu algorytmów AI.
- Dostęp do monitorowania wskaźników biznesowych poprzez dashboard administracyjny.

Założenia techniczne

Architektura systemu opiera się na następujących paradygmatach:

- **Konteneryzacja:** Izolacja usług (backend, bazy danych) w kontenerach Docker.
- **Wielomodelowe składowanie danych:** Wykorzystanie relacyjnych baz PostgreSQL dla danych strukturalnych (zamówienia, produkty) oraz dokumentowych baz MongoDB dla danych o zmiennej strukturze (opinie, profile użytkowników).
- **Komunikacja:** Centralizacja ruchu zewnętrznego przez API Gateway oraz wewnętrzna komunikacja mikroservisów w ramach sieci mostkowej.

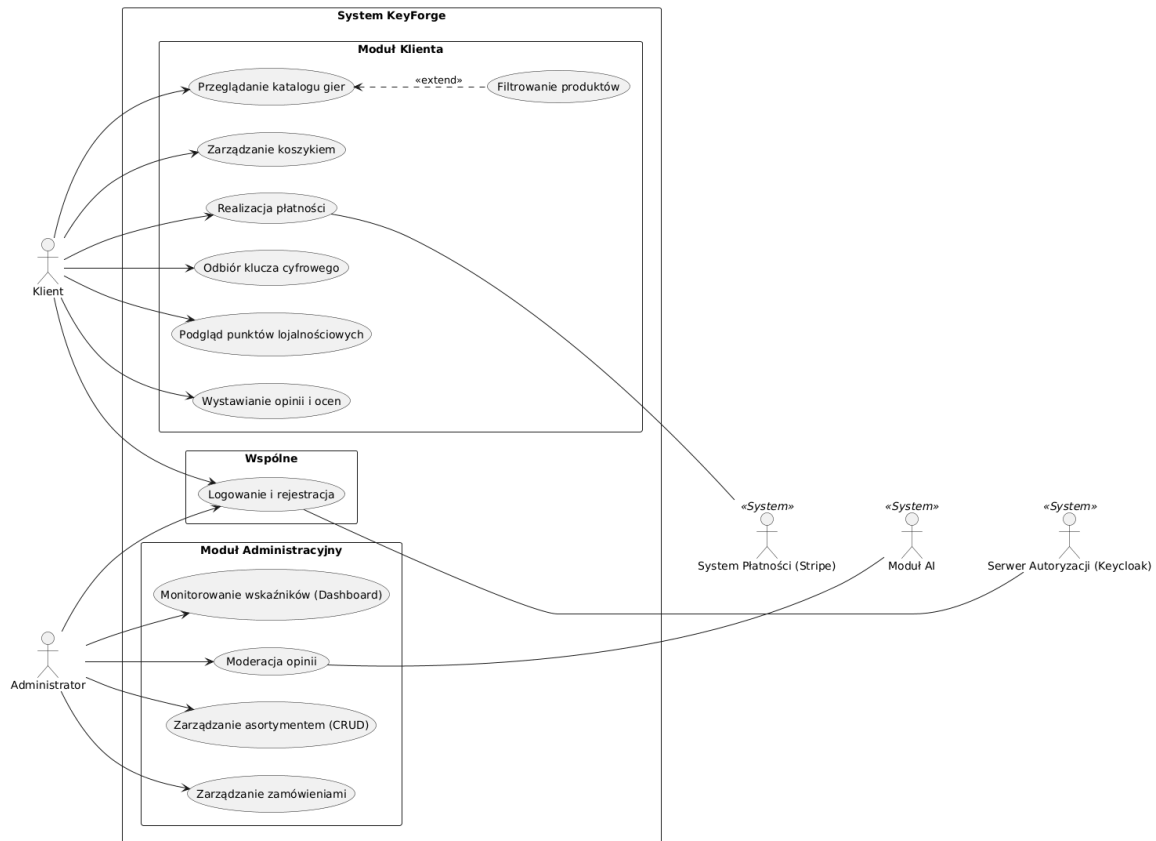
Kluczowe Funkcjonalności

- **Zarządzanie Użytkownikami:** Rejestracja, logowanie (w tym OAuth2 Google) oraz system poziomów lojalnościowych „KeyPoints”.
- **Katalog Produktów:** Interaktywna lista gier z zaawansowanym filtrowaniem po platformach (Steam, Origin) oraz kategoriach.
- **System Transakcyjny:** Obsługa płatności przez Stripe API (BLIK, karta) oraz natychmiastowe wydawanie kluczy po zakupie.
- **Moduł AI:** Automatyczna moderacja opinii oraz generowanie streszczeń recenzji użytkowników.
- **Panel Administratora:** Zarządzanie ofertami, zamówieniami oraz bazą użytkowników.

Diagramy Systemowe

Diagram przypadków użycia (UML)

Na rysunku 1 przedstawiono diagram przypadków użycia, który obrazuje zakres funkcjonalny systemu KeyForge w podziale na moduły klienta oraz administratora, wraz z uwzględnieniem integracji z systemami zewnętrznymi.



Rysunek 1: Diagram przypadków użycia systemu KeyForge.

Charakterystyka interakcji:

- **Proces zakupowy:** Klient inicjuje proces poprzez przeglądanie katalogu, a finalizuje go w module płatności, który komunikuje się z bramką Stripe.
- **Zarządzanie treścią:** Administrator posiada uprawnienia do edycji bazy produktów PostgreSQL oraz nadzoruje opinie w bazie MongoDB, gdzie system AI automatycznie oznacza recenzje wymagające uwagi.
- **Bezpieczeństwo:** Większość interakcji wymaga autoryzacji i są zabezpieczone przez Keycloak, co na diagramie odzwierciedla relacja przypadku „Logowanie i rejestracja”.

Diagram Sekwencji

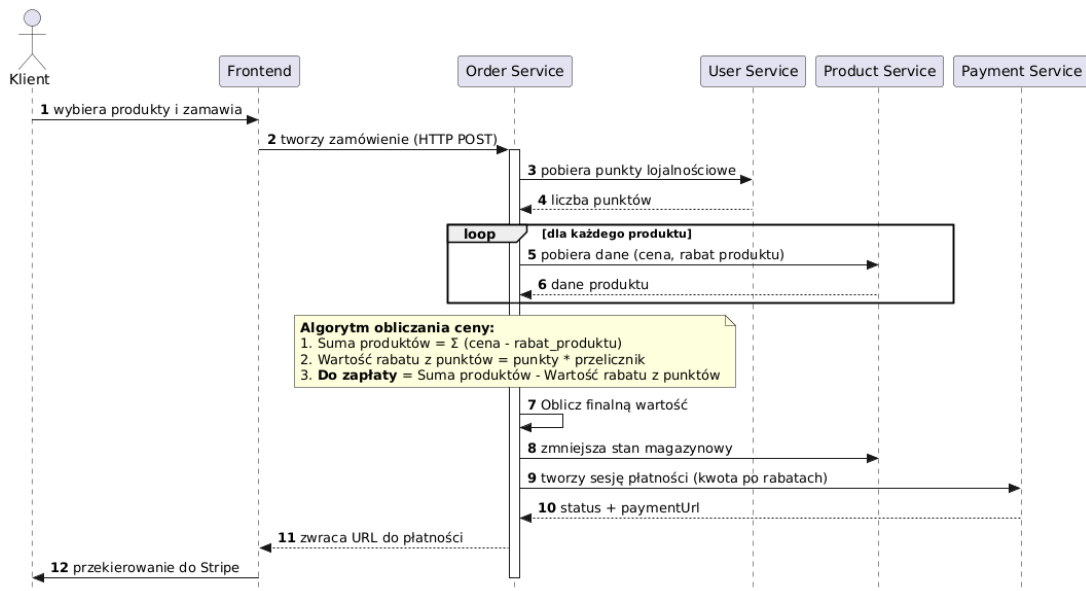
Prezentowany diagram sekwencji (Rys. 2) ilustruje interakcje pomiędzy użytkownikiem a poszczególnymi komponentami systemu podczas procesu zakupu produktów. Całość procesu można podzielić na cztery główne etapy:

1. **Inicjalizacja i agregacja danych:** Proces rozpoczyna się od wysłania żądania HTTP POST przez *Frontend* do serwisu *Order Service*. W pierwszej kolejności system komunikuje się z *User Service*, aby pobrać aktualny stan punktów lojalnościowych klienta.
2. **Weryfikacja produktów:** W pętli (*loop*), dla każdego elementu w koszyku, *Order Service* odpytuje *Product Service* o szczegółowe dane, takie jak cena jednostkowa oraz specyficzne rabaty przypisane do danego towaru.
3. **Logika obliczeniowa:** Po zebraniu danych, *Order Service* wykonuje wewnętrzne obliczenia biznesowe. Finalna kwota do zapłaty wyznaczana jest według algorytmu:

$$K = \sum(C_i - R_{pi}) - (P \cdot L) \quad (1)$$

gdzie: K to kwota końcowa, C_i to cena produktu, R_{pi} to rabat produktu, a $(P \cdot L)$ to wartość zniżki wynikająca z punktów lojalnościowych.

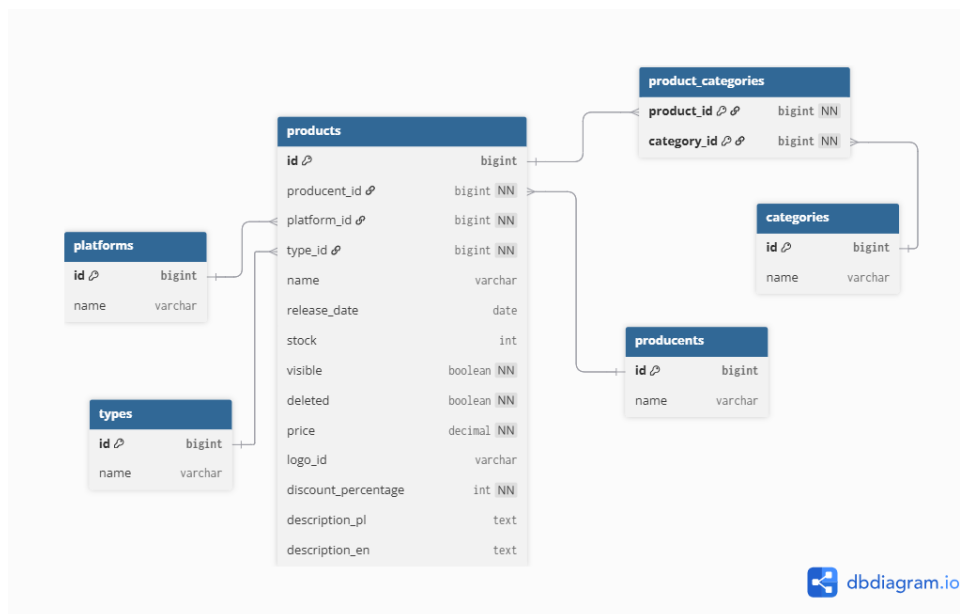
4. **Finalizacja i płatność:** Po poprawnym obliczeniu wartości, system aktualizuje stany magazynowe w *Product Service* i inicjuje sesję w zewnętrznym systemie płatności *Stripe*. Proces kończy się przekazaniem klientowi adresu URL do sfinalizowania transakcji.



Rysunek 2: Diagram sekwencji procesu składania zamówienia w architekturze mikroserysowej

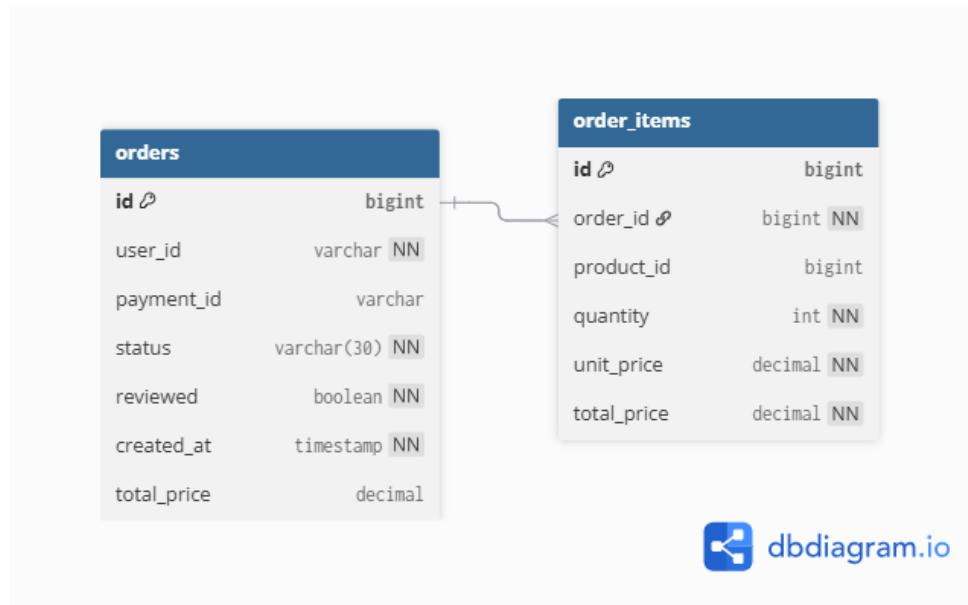
Diagram Związków Encji (ERD)

Na rysunku 3 przedstawiono diagram związków encji dla mikroserwisu odpowiedzialnego za zarządzanie produktami. Diagram ten ilustruje strukturę relacyjnej bazy danych PostgreSQL oraz zależności pomiędzy encjami reprezentującymi produkty, kategorie, platformy, typy oraz producentów. Zaprojektowany model danych umożliwia elastyczne zarządzanie ofertą produktową, jej kategoryzacją oraz powiązaniem z innymi elementami domeny systemu.



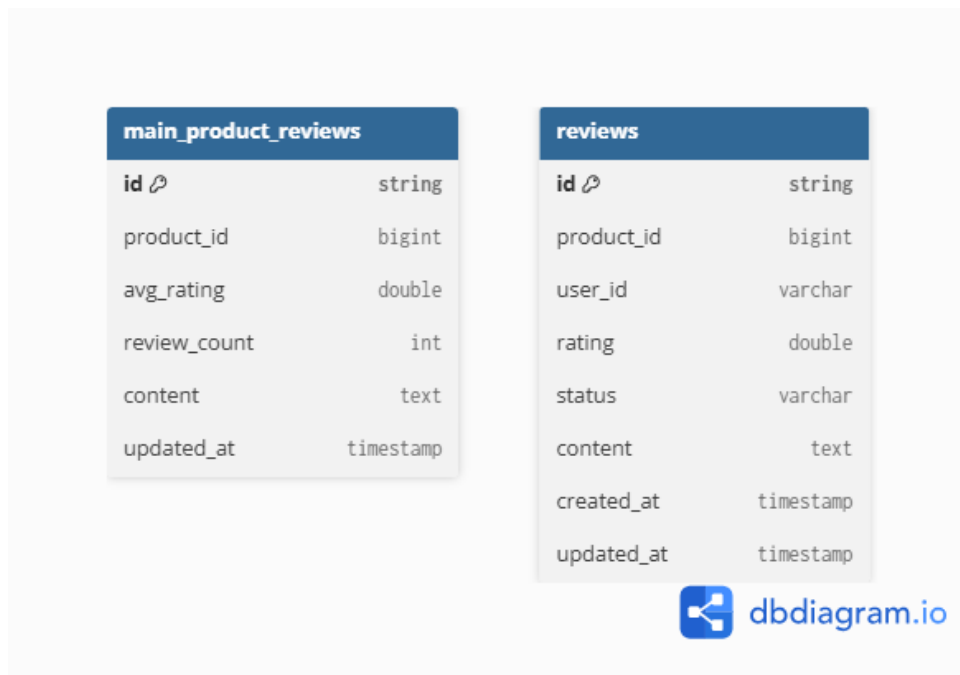
Rysunek 3: Model relacyjny danych mikroserwisu produktów – struktura bazy PostgreSQL

Na rysunku 4 zaprezentowano diagram związków encji dla mikroservisu obsługującego proces składania i realizacji zamówień. Model ten obejmuje encje reprezentujące zamówienia oraz ich pozycje, a także relacje umożliwiające odwzorowanie struktury koszyka zakupowego oraz cyklu życia zamówienia. Dane przechowywane są w relacyjnej bazie danych PostgreSQL, co zapewnia spójność transakcyjną oraz integralność danych w obrębie mikroservisu.



Rysunek 4: Model relacyjny danych mikroservisu zamówień – struktura bazy PostgreSQL

Na rysunku 5 przedstawiono model danych dla mikroserwisu odpowiedzialnego za obsługę recenzji produktów, zaimplementowanego z wykorzystaniem bazy danych MongoDB. W przeciwieństwie do pozostałych mikroserwisów, które wykorzystują relacyjny model danych, mikroserwis recenzji opiera się na modelu dokumentowym, co umożliwia elastyczne przechowywanie treści tekstowych oraz efektywne przetwarzanie danych o zmiennej strukturze.



Rysunek 5: Model dokumentowy danych mikroserwisu recenzji – struktura bazy MongoDB

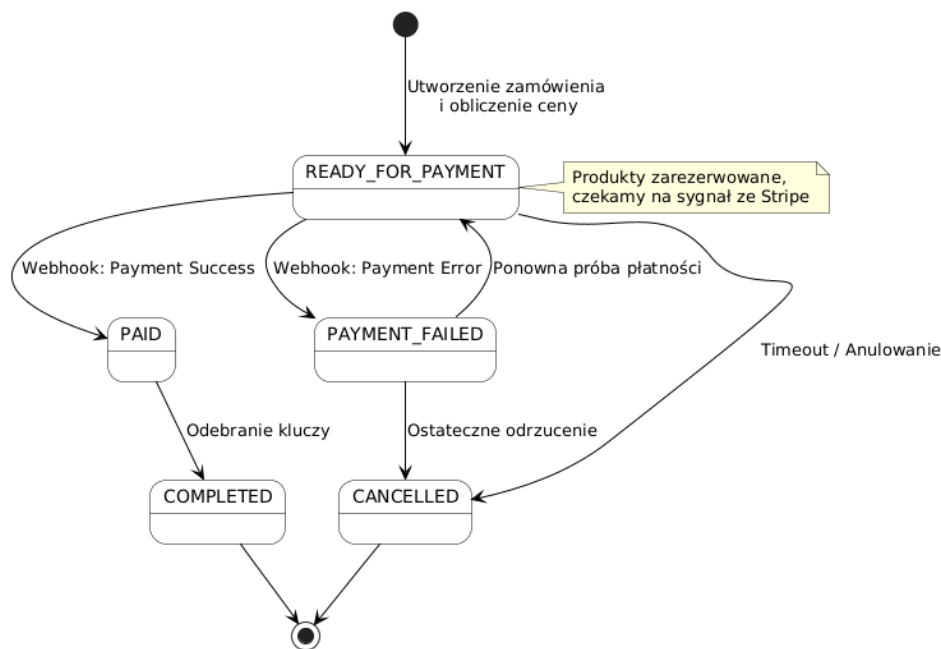
Kolekcja *reviews* przechowuje pojedyncze recenzje wystawiane przez użytkowników, zawierające m.in. identyfikator produktu, identyfikator użytkownika, ocenę, treść recenzji oraz jej status moderacyjny. Zastosowanie enumeracji *ReviewStatus*, która zawiera stany *PENDING*, *APPROVED*, *REJECTED* umożliwia kontrolę procesu akceptacji recenzji przed ich publikacją. Kolekcja *main_product_reviews* pełni natomiast rolę dokumentu agregującego dane statystyczne dotyczące danego produktu, takie jak średnia ocena oraz liczba recenzji, co pozwala na ograniczenie kosztownych operacji agregujących wykonywanych dynamicznie na dużych zbiorach danych.

Zastosowana architektura systemu opiera się na podejściu mikroserwisowym, w którym każdy mikroserwis posiada własną, niezależną bazę danych, dostosowaną do jego specyficznych wymagań funkcjonalnych. W konsekwencji relacje pomiędzy danymi należącymi do różnych mikroserwisów nie są realizowane w postaci fizycznych kluczy obcych na poziomie bazy danych. Powiązania te mają charakter logiczny i są utrzymywane poprzez identyfikatory obiektów przekazywane pomiędzy mikroserwisami za pośrednictwem interfejsów komunikacyjnych.

Diagram Przejść Stanów (STD)

Na Rysunku 6 przedstawiono automat stanów zarządzający procesem zakupu. System został zaprojektowany w sposób odporny na błędy płatności, co odzwierciedlają poniższe zależności:

- **Inicjalizacja:** Zamówienie rozpoczyna bieg w stanie `READY_FOR_PAYMENT` po pomyślnej rezerwacji produktów w magazynie.
- **Obsługa płatności:** Stan ten może zakończyć się sukcesem (`PAID`) lub błędem (`PAYMENT_FAILED`). W przypadku błędu, system umożliwia użytkownikowi ponowną próbę transakcji, co zapobiega porzucaniu koszyków.
- **Finalizacja i Anulowanie:** Po opłaceniu i wydaniu kluczy gier, zamówienie przechodzi w stan terminalny `COMPLETED`. W przypadku przekroczenia czasu oczekiwania lub ostatecznego odrzucenia płatności, stanem końcowym jest `CANCELLED`, co skutkuje automatycznym zwolnieniem blokady towarów.



Rysunek 6: Cykl życia zamówienia w oparciu o stany `OrderStatus`

Implementacja Warstwy Danych

System KeyForge wykorzystuje podejście **Code-First** do definiowania struktury baz danych. Dzięki zastosowaniu **Spring Data JPA** oraz **Hibernate**, schemat bazy danych PostgreSQL jest automatycznie generowany i synchronizowany na podstawie klas encyjnych w języku Java.

Mapowanie obiektowo-relacyjne (ORM)

Wszystkie encje w mikroservisach *product-service* oraz *order-service* zostały zmapowane na tabele relacyjne przy użyciu adnotacji JPA. Pozwoliło to na precyzyjne zdefiniowanie ograniczeń (*constraints*) bezpośrednio w kodzie źródłowym.

- **@Column(nullable = false):** Zapewnia integralność danych poprzez wymuszenie wartości niepustych na poziomie bazy.
- **@JoinColumn:** Definiuje fizyczne powiązania i klucze obce między tabelami (np. relacja Produkt – Kategoria).
- **Encje MongoDB:** W mikroservisie recenzji zastosowano mapowanie dokumentowe przy użyciu adnotacji **@Document**, co pozwala na elastyczne przechowywanie danych NoSQL.

Tryb połączenia z bazą danych

Aplikacja łączy się z bazami danych PostgreSQL i MongoDB uruchomionymi w kontenerach Docker. W środowisku deweloperskim wykorzystywany jest tryb **update** (`hibernate.ddl-auto`), co pozwala na dynamiczne dostosowywanie schematu bazy bez utraty danych testowych. Połączenie realizowane jest poprzez sterowniki JDBC (dla PostgreSQL) oraz natywne sterowniki Reactive Streams (dla MongoDB).

Listing 1: Przykład implementacji ograniczeń w encji Product (Code-First)

```
@NoArgsConstructor
@AllArgsConstructor
@Getter
@Setter
@Entity
@Table(name="products")
public class Product {
    @PrePersist
    public void prePersist() {
        if (discountPercentage == null) discountPercentage = 0;
        if (visible == null) visible = true;
        if (deleted == null) deleted = false;
    }

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
```

```

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "producent_id", nullable = false)
    private Producent producent;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "platform_id", nullable = false)
    private Platform platform;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "type_id", nullable = false)
    private Type type;

    @ManyToMany
    @JoinTable(
        name = "product_categories",
        joinColumns = @JoinColumn(name = "product_id"),
        inverseJoinColumns = @JoinColumn(name = "category_id")
    )
    private Set<Category> categories = new HashSet<>();

    private String name;
    private LocalDate releaseDate;
    private Integer stock;

    @Column(nullable = false, columnDefinition = "BOOLEAN DEFAULT FALSE")
    private Boolean visible;

    @Column(nullable = false, columnDefinition = "BOOLEAN DEFAULT FALSE")
    private Boolean deleted;

    @Column(nullable = false)
    private BigDecimal price;

    @Column(nullable = true)
    private String logoId;

    @Column(nullable = false)
    private Integer discountPercentage = 0;

    @Column(columnDefinition = "TEXT")
    private String descriptionPl;

    @Column(columnDefinition = "TEXT")
    private String descriptionEn;
}

```

Dokumentacja interfejsów API (Swagger/OpenAPI)

W procesie wytwarzania systemu *KeyForge*, kluczowym aspektem było zapewnienie przejrzystej i zawsze aktualnej dokumentacji punktów końcowych (endpoints) dla wszystkich mikroservisów. W tym celu wykorzystano standard **OpenAPI 3.0** oraz narzędzie **Swagger UI**.

Każdy mikroservis w systemie (m.in. `order-service`, `product-service`, `user-service`) posiada zintegrowaną bibliotekę *springdoc-openapi*, która automatycznie generuje interaktywną stronę dokumentacji.

Dostęp do dokumentacji Swagger dla poszczególnych usług odbywa się poprzez dedykowane porty zmapowane na maszynę lokalną. Wykaz wszystkich dostępnych portów, na których wystawione są interfejsy dokumentacji, znajduje się w sekcji **Konfiguracja Docker Compose** w parametrach `ports` poszczególnych serwisów.

Dokumentowanie interfejsów za pomocą Swaggera pozwoliło na:

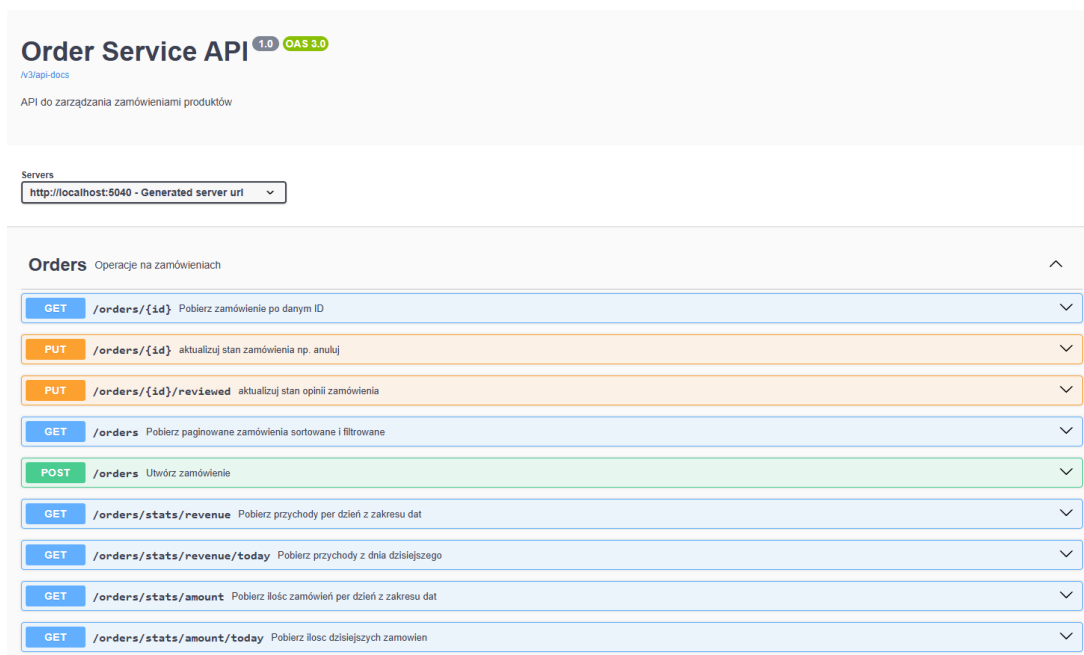
- **Ujednolicenie komunikacji:** Dokumentacja zapewnia pełną spójność wymiany danych między warstwą prezentacji (React.js) a mikroservisami, oferując precyzyjny wgląd w wymagane parametry zapytań oraz strukturę zwracanych obiektów JSON.
- **Testowanie bez narzędzi zewnętrznych:** Swagger pozwala na wysyłanie żądań testowych (np. GET, POST) bezpośrednio z poziomu przeglądarki, co znacząco przyspieszyło debugowanie logiki biznesowej, dopóki nie dodano autoryzacji na endpointy.
- **Walidację kontraktów:** Automatyczne generowanie schematów (Schemas) gwarantuje, że dokumentacja jest zawsze zgodna z faktycznym kodem źródłowym aplikacji.

Weryfikacja i testowanie interfejsów API

W celu zapewnienia wysokiej jakości usług oraz poprawnej integracji między mikroservisami, proces wytwórczy wspierany był przez regularne testy interfejsów programistycznych. Poniżej przedstawiono dokumentację punktów końcowych oraz przykłady wywołań wykonane przy użyciu narzędzi **Swagger UI** oraz **Insomnia**.

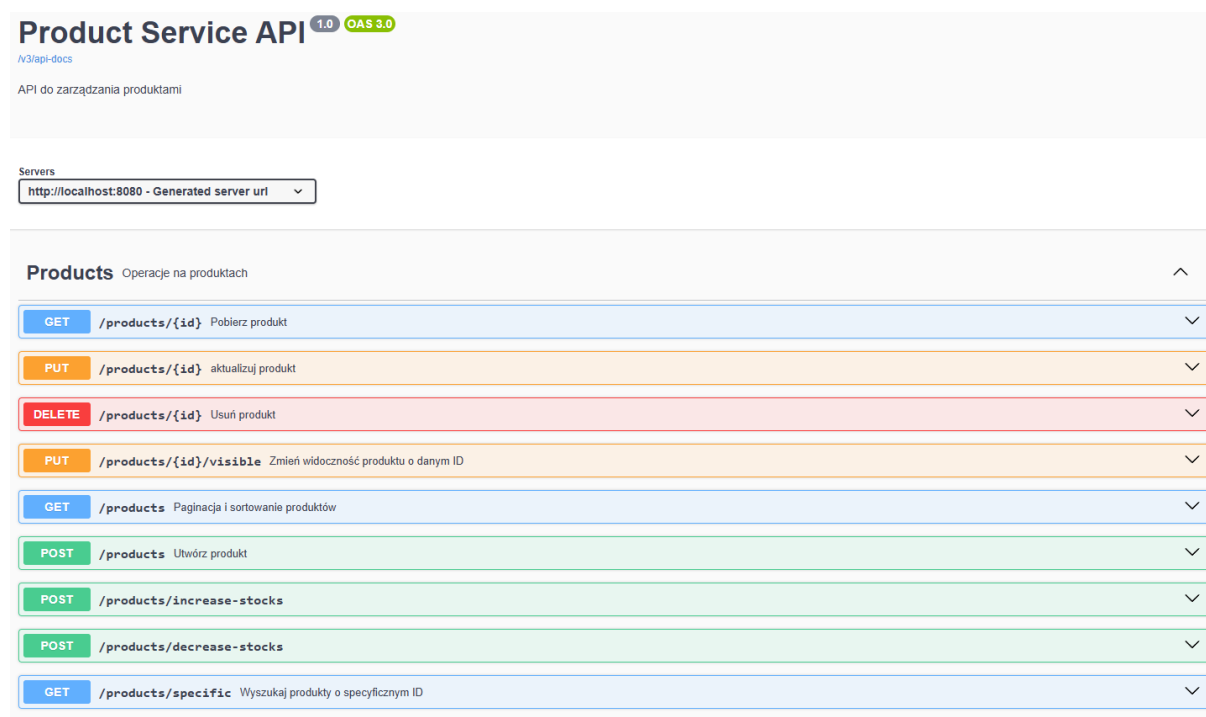
Interfejs Swagger (OpenAPI)

Swagger służy jako żywa dokumentacja techniczna systemu. Pozwala na szybki przegląd dostępnych metod oraz struktur danych (DTO) bez konieczności analizy kodu źródłowego.



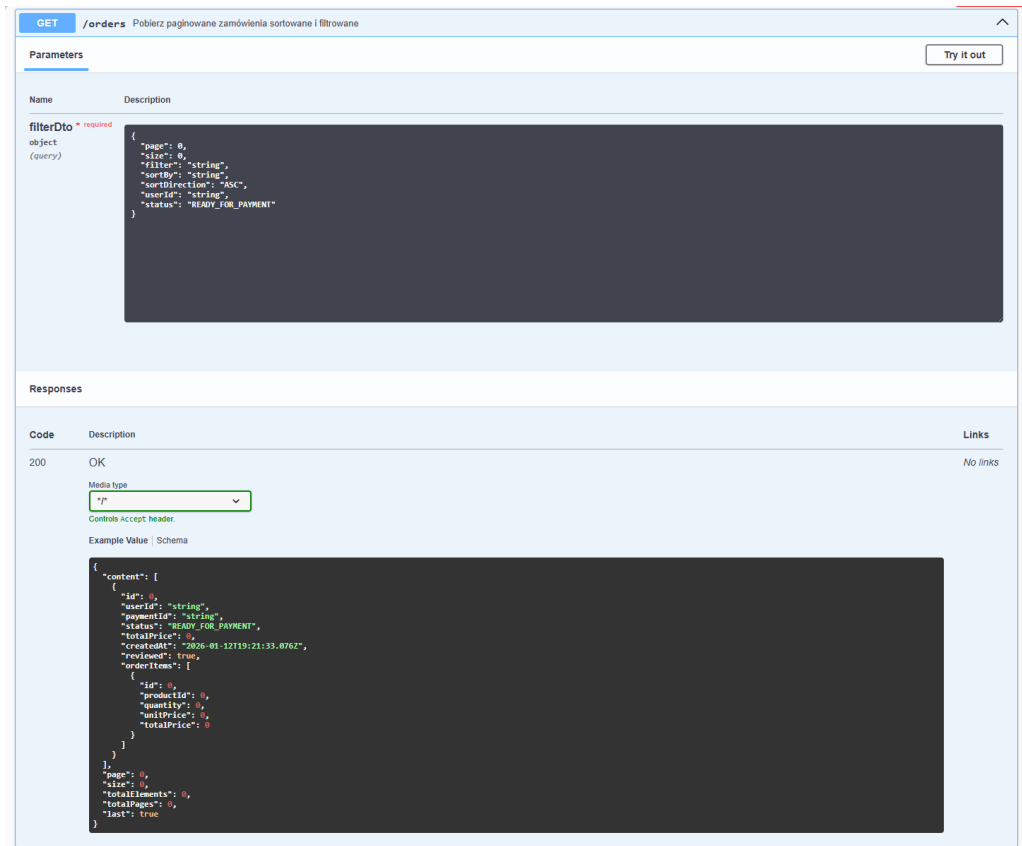
Rysunek 7: Lista dostępnych punktów końcowych mikroserwisu zamówień w Swagger UI.

Na Rysunku 8 zaprezentowano przykładowy widok interfejsu Swagger dla mikroserwisu produktów (*Product Service*), prezentujący dostępne operacje oraz modele danych.



Rysunek 8: Interfejs Swagger UI prezentujący punkty końcowe mikroserwisu Product Service.

Na rysunku 9 przedstawiono szczegółową specyfikację punktu końcowego GET /orders, który obsługuje filtrowanie oraz paginację wyników, co jest kluczowe dla wydajności przy dużej liczbie transakcji.



The image shows a Swagger UI specification for the GET /orders endpoint. The title bar indicates the method is GET and the path is /orders, with a subtitle 'Pobierz paginowane zamówienia sortowane i filtrowane'. A 'Try it out' button is present in the top right.

The 'Parameters' section lists a required parameter 'filterDto' of type 'object' (query). The description for this parameter is a JSON object with the following schema:

```
{
  "page": 0,
  "size": 0,
  "filter": "string",
  "sortBy": "string",
  "sortDirection": "ASC",
  "userId": "string",
  "status": "READY_FOR_PAYMENT"
}
```

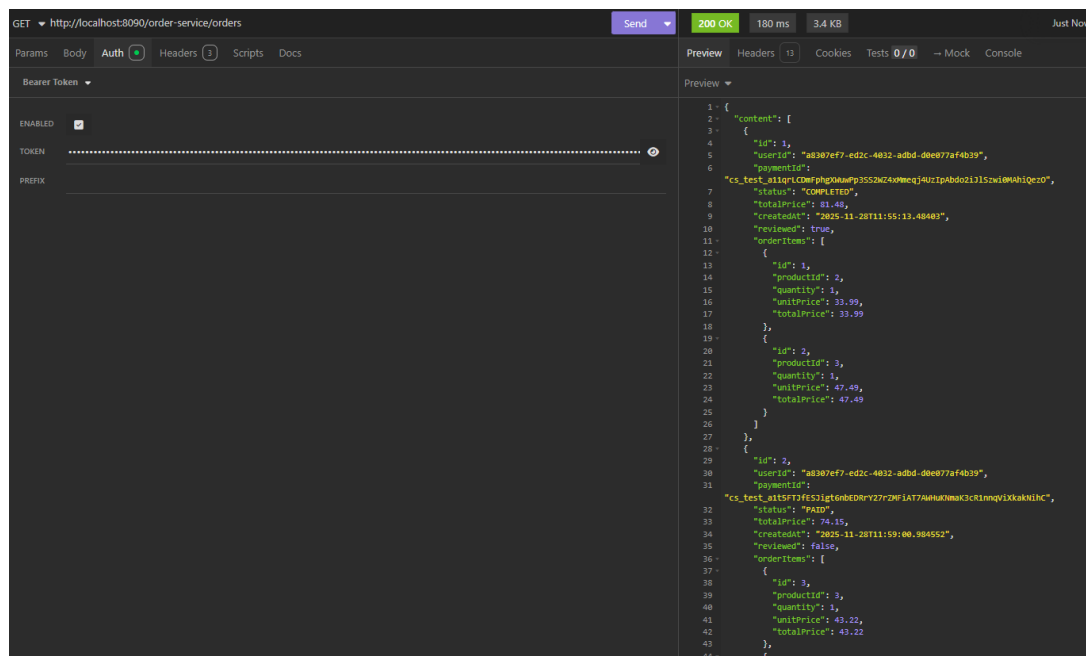
The 'Responses' section shows a 200 OK response. The media type is '*/'. Below the media type, there is a 'Controls Accept header' section and an 'Example Value' section. The example value is a JSON object representing the response structure:

```
{
  "content": [
    {
      "id": 0,
      "userId": "string",
      "paymentId": "string",
      "status": "READY_FOR_PAYMENT",
      "totalPrice": 0,
      "createdAt": "2026-01-12T19:21:33.076Z",
      "reviewed": true,
      "orderItems": [
        {
          "id": 0,
          "productId": 0,
          "quantity": 0,
          "unitPrice": 0,
          "totalPrice": 0
        }
      ]
    }
  ],
  "page": 0,
  "size": 0,
  "totalElements": 0,
  "totalPages": 0,
  "last": true
}
```

Rysunek 9: Specyfikacja parametrów paginacji i struktury odpowiedzi dla zamówień.

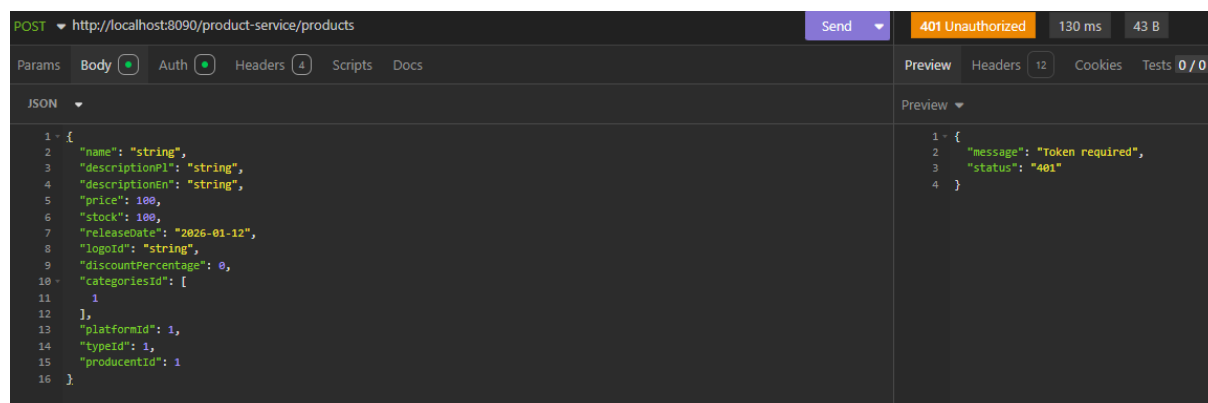
Testy w środowisku Insomnia

Narzędzie Insomnia zostało wykorzystane do testowania rzeczywistych scenariuszy przepływu danych, w tym weryfikacji mechanizmów bezpieczeństwa oraz poprawności odpowiedzi serwera.

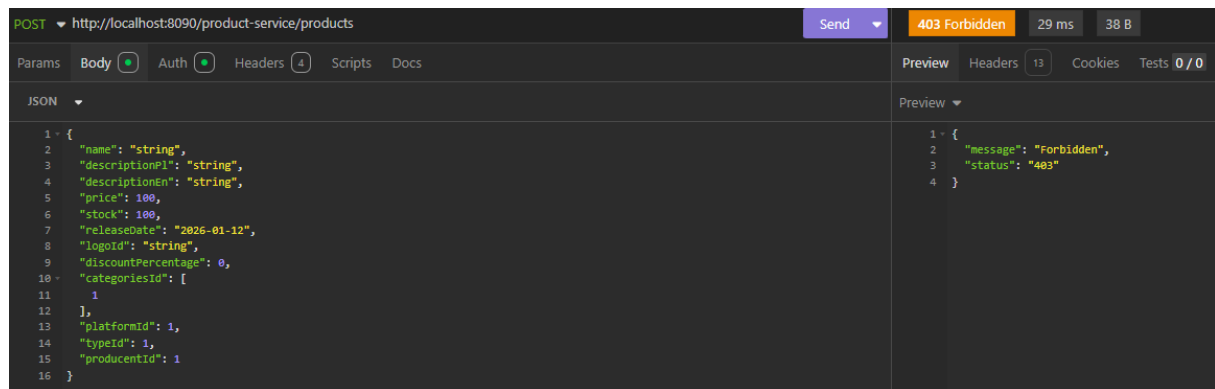


Rysunek 10: Pomyślne pobranie listy zamówień przy użyciu autoryzacji Bearer Token.

Ważnym elementem testów była weryfikacja obsługi błędów i uprawnień. System poprawnie identyfikuje brak poświadczeń (kod 401 Unauthorized) oraz próby wykonania akcji administracyjnych przez zwykłych użytkowników (kod 403 Forbidden), co przedstawiono na poniższych zrzutach ekranu.



Rysunek 11: Weryfikacja bezpieczeństwa: Odpowiedź serwera 401 Unauthorized przy braku tokena.



Rysunek 12: Weryfikacja uprawnień: Odpowiedź 403 Forbidden przy próbie modyfikacji produktu przez użytkownika bez roli administratora.

Architektura Konteneryzacji

System KeyForge opiera się na konteneryzacji wszystkich komponentów, co zapewnia skalowalność i spójność środowisk. Obrazy mikroservisów są budowane i przechowywane w **GitHub Container Registry**.

Zasady Mapowania Portów

Uwaga: W poniższej konfiguracji zastosowano mapowanie portów poszczególnych kontenerów bezpośrednio na maszynę lokalną (hosta). Zastrzega się, że rozwiązanie to służy wyłącznie celom deweloperskim, aby ułatwić dostęp do narzędzi takich jak dokumentacja Swagger czy bezpośredni wgląd w bazy danych. W środowisku produkcyjnym cała komunikacja z zewnątrz powinna odbywać się wyłącznie za pośrednictwem serwisu **api-gateway**.

Konfiguracja Docker Compose

Poniższy kod definiuje pełne środowisko uruchomieniowe systemu, uwzględniając podział na sieci oraz zależności między usługami. Dla lepszej czytelności konfiguracja została podzielona na logiczne sekcje.

```
1 version: '3.9'
2
3 services:
4     # =====
5     # INFRASTRUKTURA UWIERZYTELNIANIA
6     # =====
7     postgres-keycloak:
8         container_name: postgres-keycloak
9         image: postgres
10        environment:
11            POSTGRES_USER: "keycloak"
12            POSTGRES_PASSWORD: "keycloak"
13            POSTGRES_DB: "keycloak"
14        networks:
15            - keyforge-network
16
17    keycloak:
18        image: 'quay.io/keycloak/keycloak:latest'
19        container_name: keycloak
20        command: ["start-dev"]
21        environment:
22            KC_DB: "postgres"
23            KC_DB_URL_HOST: "postgres-keycloak"
24            KC_DB_URL_DATABASE: "keycloak"
25        ports:
26            - "5000:8080" # Port deweloperski
27        depends_on:
28            - postgres-keycloak
29        networks:
30            - keyforge-network
```

```
1     # =====
2     # SERWIS UZYTEKOWNIKOW (MongoDB)
3     # =====
4     mongo-users:
5         container_name: mongo-users
6         image: mongo
7         environment:
8             MONGO_INITDB_ROOT_USERNAME: "mongo-users"
9             MONGO_INITDB_ROOT_PASSWORD: "mongo-users"
10            MONGO_INITDB_DATABASE: "usersdb"
11        networks:
12            - keyforge-network
```

```

13
14 user-service:
15   container_name: user-service
16   build:
17     context: ./user-service
18     dockerfile: Dockerfile
19   ports:
20     - "5020:5020"
21   environment:
22     MONGO_HOST: "mongo-users"
23     KEYCLOAK_URL: "http://keycloak:8080"
24   depends_on:
25     - mongo-users
26   networks:
27     - keyforge-network
28
29 # =====
30 # SERWIS PRODUKTOW (PostgreSQL)
31 # =====
32 postgres-products:
33   container_name: postgres-products
34   image: postgres
35   environment:
36     POSTGRES_USER: "postgres-products"
37     POSTGRES_PASSWORD: "postgres-products"
38     POSTGRES_DB: "postgres-products"
39   networks:
40     - keyforge-network
41
42 product-service:
43   container_name: product-service
44   build:
45     context: ./product-service
46   ports:
47     - "5050:5050"
48   environment:
49     DB_HOST: "postgres-products"
50   depends_on:
51     - postgres-products
52   networks:
53     - keyforge-network

```

```

1 # =====
2 # SERWIS ZAMOWIEN (PostgreSQL)
3 # =====
4 postgres-orders:
5   container_name: postgres-orders
6   image: postgres
7   environment:
8     POSTGRES_USER: "postgres-orders"
9     POSTGRES_PASSWORD: "postgres-orders"
10    POSTGRES_DB: "postgres-orders"
11   networks:
12     - keyforge-network
13
14 order-service:
15   container_name: order-service
16   build:
17     context: ./order-service
18   ports:
19     - "5040:5040"
20   environment:
21     PRODUCT_SERVICE_URL: "http://product-service:5050"
22     DB_HOST: "postgres-orders"
23   depends_on:
24     - postgres-orders
25   networks:
26     - keyforge-network
27
28 # =====
29 # SERWIS PLATNOSCI (Stripe API)
30 # =====

```

```

31 payment-service:
32   container_name: payment-service
33   build:
34     context: ./payment-service
35   ports:
36     - "5070:5070"
37   environment:
38     ORDER_SERVICE_URL: "http://order-service:5040"
39   depends_on:
40     - order-service
41   networks:
42     - keyforge-network

```

```

1  # =====
2  # SERWIS RECENZJI (MongoDB)
3  # =====
4  mongo-review:
5    container_name: mongo-review
6    image: mongo
7    networks:
8      - keyforge-network
9
10 review-service:
11   container_name: review-service
12   build:
13     context: ./review-service
14   ports:
15     - "5080:5080"
16   environment:
17     AI_SERVICE_URL: "http://ai-service:5030"
18     MONGO_HOST: "mongo-review"
19   depends_on:
20     - mongo-review
21   networks:
22     - keyforge-network
23
24  # =====
25  # SERWIS AI (Moderacja i analiza)
26  # =====
27  ai-service:
28    container_name: ai-service
29    build:
30      context: ./ai-service
31    ports:
32      - "5030:5030"
33    networks:
34      - keyforge-network

```

```

1  # =====
2  # API GATEWAY (Punkt wejsciowy)
3  # =====
4  api-gateway:
5    container_name: api-gateway
6    build:
7      context: ./api-gateway
8    ports:
9      - "8090:8090" # Port glowny aplikacji
10   environment:
11     KEYCLOAK_SERVICE_URL: "http://keycloak:8080/realms/keyforge"
12   networks:
13     - keyforge-network
14
15  # =====
16  # KONFIGURACJA SIECI
17  # =====
18  networks:
19    keyforge-network:
20      driver: bridge

```

Skonteneryzowane Usługi

Zgodnie z konfiguracją `docker-compose.yml`, system składa się z następujących kontenerów:

1. Infrastruktura Tożsamości:

- `keycloak`: Serwer autoryzacyjny zarządzający dostępem i tokenami JWT.
- `postgres-keycloak`: Dedykowana baza danych PostgreSQL dla Keycloak.

2. Usługi Biznesowe (Spring Boot):

- `user-service`: Zarządzanie profilami (Baza: MongoDB).
- `product-service`: Katalog gier, kategorie i ceny (Baza: PostgreSQL).
- `order-service`: Obsługa koszyka i logiki zamówień (Baza: PostgreSQL).
- `payment-service`: Integracja ze Stripe i obsługa webhooków.
- `review-service`: System ocen i opinii (Baza: MongoDB).
- `ai-service`: Integracja z zewnętrznymi modelami AI dla moderacji treści.

3. Warstwa Sieciowa i Storage:

- `api-gateway`: Centralny punkt wejścia oparty na Spring Cloud Gateway, realizujący routing do odpowiednich serwisów.
- `mongo-users` & `mongo-review`: Kontenery bazodanowe NoSQL dla danych użytkowników i opinii.
- `postgres-products` & `postgres-orders`: Kontenery bazodanowe SQL dla danych strukturalnych.

Sieć i Komunikacja

Wszystkie kontenery pracują w ramach zdefiniowanej sieci mostkowej (*bridge network*) o nazwie `keyforge-network`. Usługi komunikują się wewnętrznie przy użyciu nazw kontenerów (np. `http://product-service:5050`), a dostęp zewnętrzny realizowany jest wyłącznie przez `api-gateway` na porcie 8090.

Konfiguracja i Uruchomienie Systemu

Niniejszy rozdział opisuje procedurę przygotowania środowiska oraz kroki niezbędne do poprawnego uruchomienia kompletnego ekosystemu mikroservisów *KeyForge*.

Wymagania systemowe

Do poprawnego zbudowania i uruchomienia wszystkich komponentów wymagane jest posiadanie następujących narzędzi:

- **Docker & Docker Compose:** Narzędzia do orkiestracji kontenerów.
- **Java 21 (OpenJDK):** Wymagana do ewentualnej kompilacji źródeł poza kontenerem.
- **Gradle:** System budowania wykorzystywany w projektach Spring Boot.

Konfiguracja zmiennych środowiskowych

Przed uruchomieniem kontenerów należy utworzyć plik `.env` w głównym katalogu projektu. Plik ten przechowuje wrażliwe dane konfiguracyjne oraz klucze dostępu do zewnętrznych usług (Stripe, Google AI).

Listing 2: Struktura pliku konfiguracyjnego `.env`

```
1 # Konfiguracja Stripe (Platnosci)
2 STRIPE_SECRET_KEY=sk_test_...
3 STRIPE_WEBHOOK_SECRET=whsec_...
4
5 # Google AI (Moderacja i analizy)
6 GOOGLE_AI_API_KEY=your_api_key_here
7
8 # Autoryzacja i CORS
9 USER_SERVICE_KEYCLOAK_SECRET=your_secret
10 CORS_ALLOWED_ORIGINS=http://localhost:5173
```

Procedura uruchomienia

Uruchomienie systemu odbywa się przy pomocy standardowych instrukcji pakietu Docker Compose. Proces ten obejmuje budowę obrazów lokalnych oraz pobranie obrazów bazowych (PostgreSQL, MongoDB, Keycloak).

1. Pobranie repozytorium:

```
git clone <repository-url>
cd keyforge-microservices
```

2. Uruchomienie usług w tle:

```
docker-compose up -d
```

3. Weryfikacja stanu kontenerów:

```
docker-compose ps
```

Punkty dostępowe

Po pomyślnym uruchomieniu, kluczowe moduły systemu są dostępne pod następującymi adresami w sieci lokalnej:

Usługa	Adres URL	Opis
API Gateway	http://localhost:8090	Punkt wejściowy do API
Keycloak Admin	http://localhost:5000	Panel zarządzania tożsamością
Frontend App	http://localhost:5173	Interfejs użytkownika (React)

Tabela 1: Zestawienie głównych punktów dostępowych systemu.

Interfejs użytkownika

Warstwa prezentacji systemu KeyForge została zaprojektowana zgodnie z paradygmatem **Single Page Application (SPA)** przy użyciu biblioteki **React.js**. Interfejs stawia na intuicyjność (User Experience) oraz pełną responsywność (**Responsive Web Design**), co umożliwia wygodne korzystanie z platformy zarówno na urządzeniach stacjonarnych, jak i mobilnych.

Panel klienta

Strona główna (Landing Page)

Strona główna systemu KeyForge (Rysunek 13) pełni rolę wizytówki platformy oraz centralnego punktu nawigacyjnego. Została zaprojektowana w ciemnej tonacji, co jest standardem estetycznym w branży gamingowej, podkreślając nowoczesny charakter serwisu.



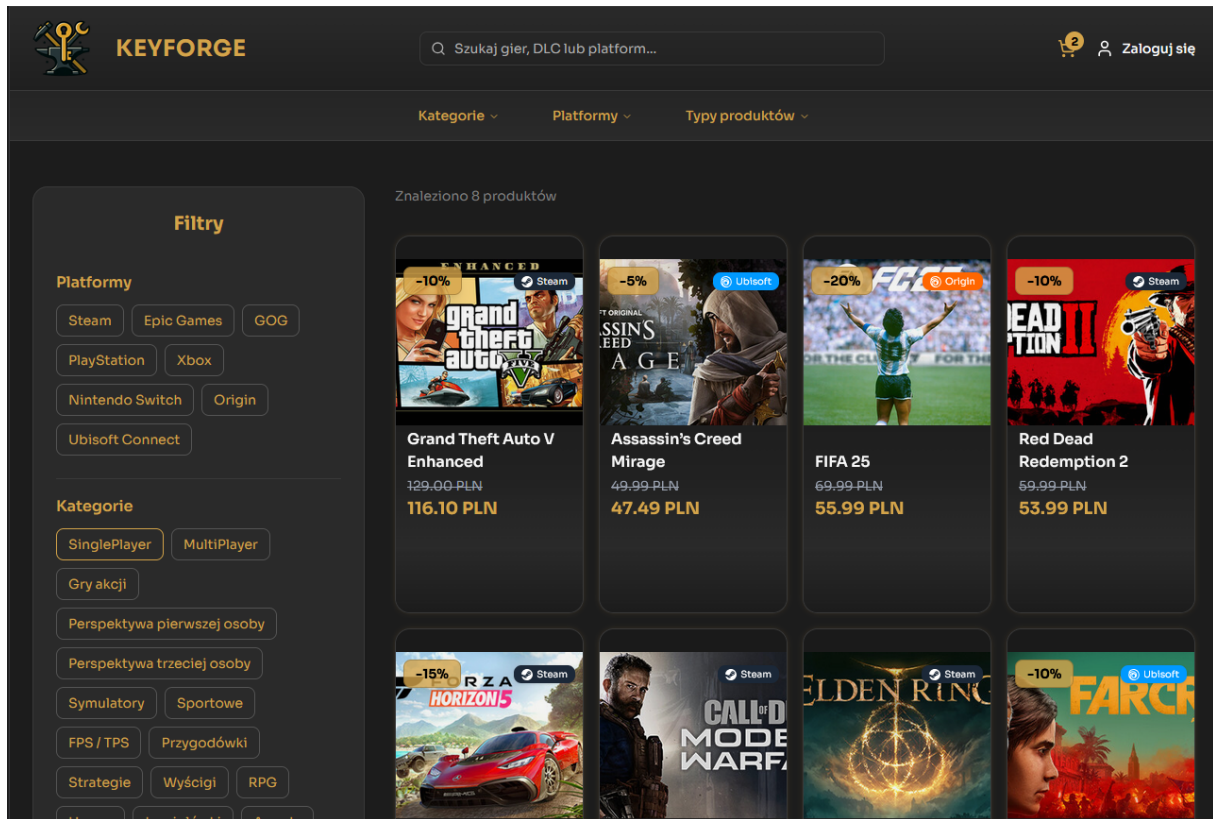
Rysunek 13: Strona główna platformy KeyForge z widocznym banerem i nawigacją.

Główne elementy interfejsu:

- **Pasek nawigacyjny (Header):** Zawiera logotyp systemu, wyszukiwarkę tekstową umożliwiającą szybkie odnalezienie gier lub platform, licznik przedmiotów w koszyku oraz przycisk logowania zintegrowany z usługą Keycloak.
- **Menu kategoryzacji:** Rozwijane listy (Kategorie, Platformy, Typy produktów) pozwalają na precyzyjne filtrowanie asortymentu poprzez komunikację z **product-service**.
- **Sekcja Hero (Hero Section):** Centralny baner z grafiką i przyciskiem *Call to Action* (CTA) „Przeglądaj gry”, który kieruje użytkownika bezpośrednio do pełnego katalogu produktów.

Moduł wyszukiwania i filtrowania

Sekcja katalogu produktów (Rysunek 14) jest kluczowym elementem interfejsu, umożliwiającym użytkownikowi sprawne poruszanie się po asortymencie sklepu. Moduł ten integruje się bezpośrednio z `product-service`, przysyłając parametry zapytania (*query parameters*) w celu dynamicznego zawężania wyników.



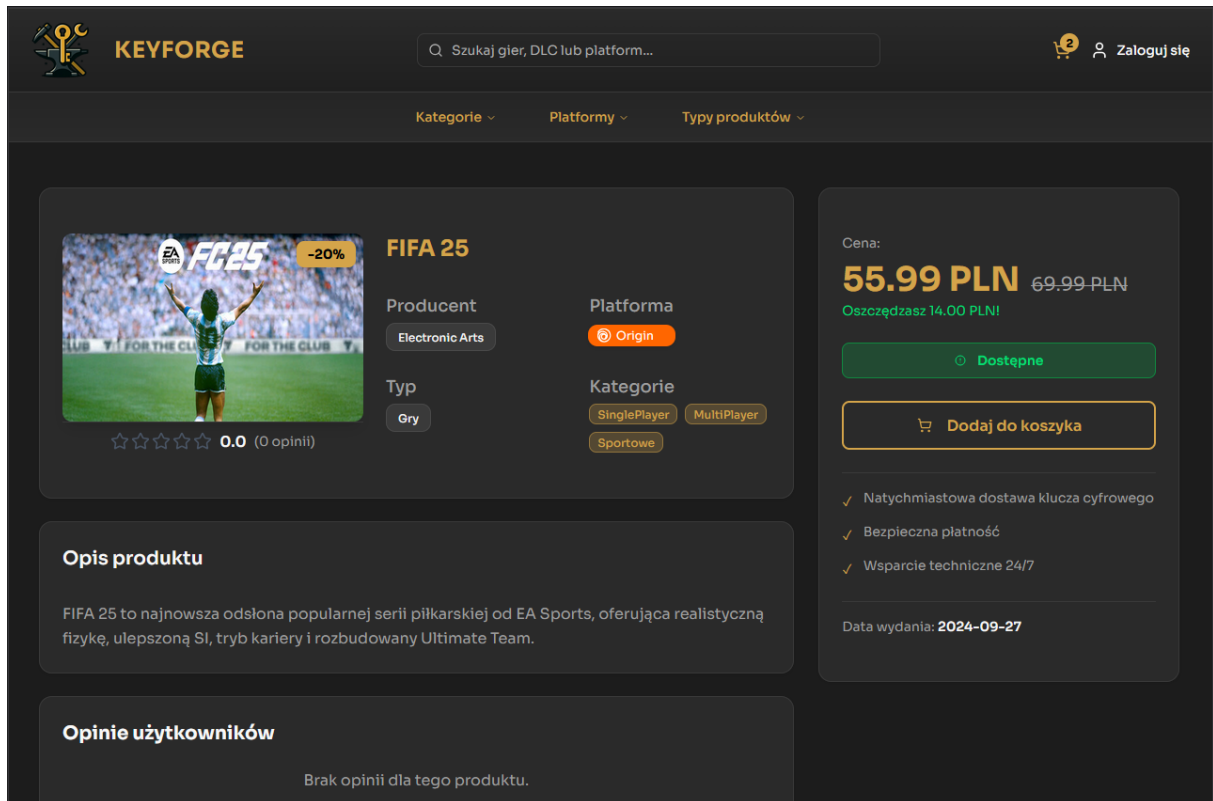
Rysunek 14: Widok katalogu z aktywnym panelem filtrów bocznych.

Główne elementy interfejsu:

- **Panel filtrów bocznych:** Pozwala na wielokrotny wybór parametrów takich jak platforma (np. Steam, Epic Games, PlayStation) oraz kategoria (np. Gry akcji, RPG, FPS). Wybór filtra wyzwała asynchroniczne żądanie do bazy PostgreSQL, odświeżając listę produktów bez przeładowania strony.
- **Karty produktów w siatce (Grid View):** Każda karta prezentuje najważniejsze metadane: tytuł, platformę docelową, cenę bazową oraz cenę po naliczonym rabacie. Wyraźne etykiety procentowe (np. -20%) informują o aktualnych promocjach.
- **Dynamiczny licznik wyników:** Nad siatką produktów znajduje się informacja o liczbie znalezionych pozycji (np. „Znaleziono 8 produktów”), co pozwala użytkownikowi na bieżąco oceniać skuteczność nałożonych filtrów.
- **Wyszukiwarka globalna:** Umieszczona w nagłówku, pozwala na przeszukiwanie bazy danych po nazwach gier oraz dodatków DLC, współpracując z systemem odpowiedzi.

Karta produktu

Widok szczegółowy produktu (Rysunek 15) stanowi kluczowy element procesu decyzyjnego użytkownika. Agreguje on dane z dwóch mikroservisów: **product-service** (podstawowe informacje, stany magazynowe), **review-service** (opinie i oceny).



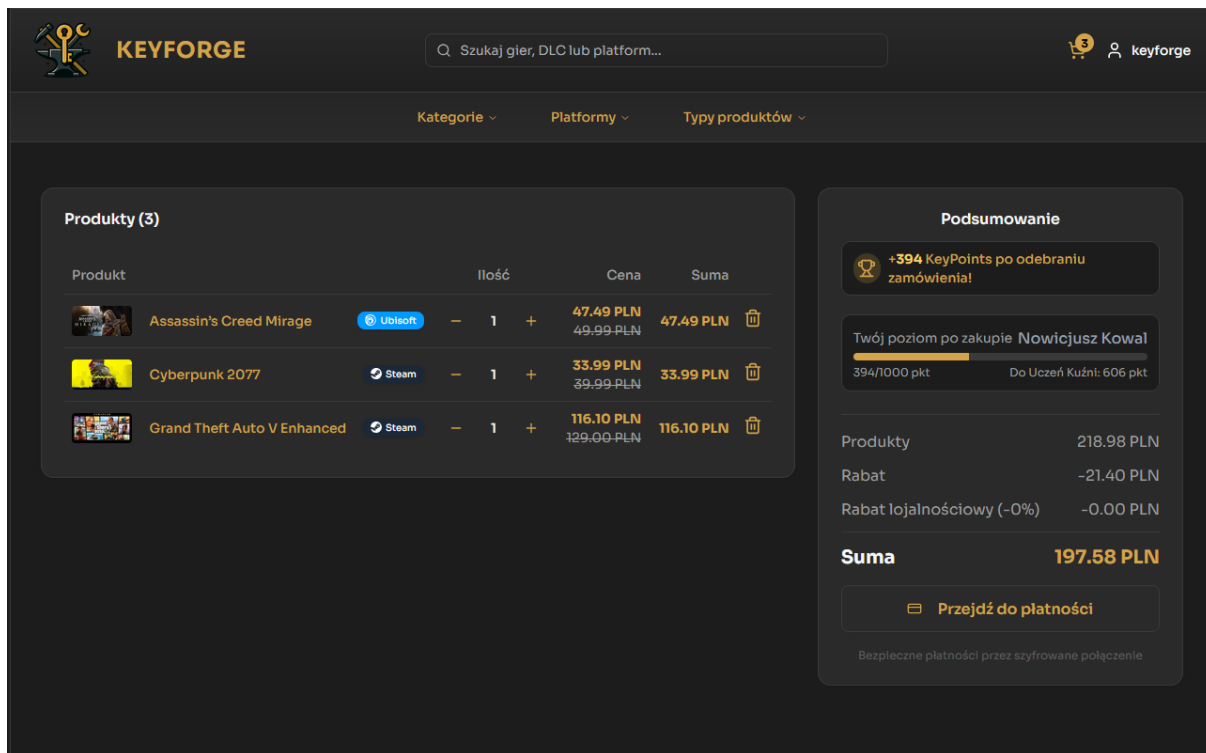
Rysunek 15: Karta produktu gry FIFA 25 w systemie KeyForge.

Główne elementy interfejsu:

- **Sekcja informacji technicznych:** Wyświetla dynamicznie generowane „tagi” dotyczące producenta, platformy (np. Origin) oraz kategorii (np. SinglePlayer, Sportowe). (obsługiwane przez **product-service**).
- **Panel zakupowy (Side Panel):** Wyraźnie wyeksponowana cena z uwzględnieniem rabatu procentowego, status dostępności produktu oraz przycisk „Dodaj do koszyka”.
- **Opis i Recenzje:** Rozbudowana sekcja tekstowa z opisem gry oraz moduł opinii użytkowników. W przypadku braku recenzji, system wyświetla dedykowany komunikat, zachęcając do wystawienia pierwszej oceny (obsługiwane przez **review-service**).
- **System ocen:** Wizualna reprezentacja średniej oceny w postaci gwiazdek, wyliczana na podstawie dokumentów z bazy MongoDB.

Koszyk zakupowy

Widok koszyka (Rysunek 16) stanowi finalny etap przygotowania transakcji. Interfejs został zaprojektowany tak, aby w sposób przejrzysty prezentować zestawienie wybranych produktów.



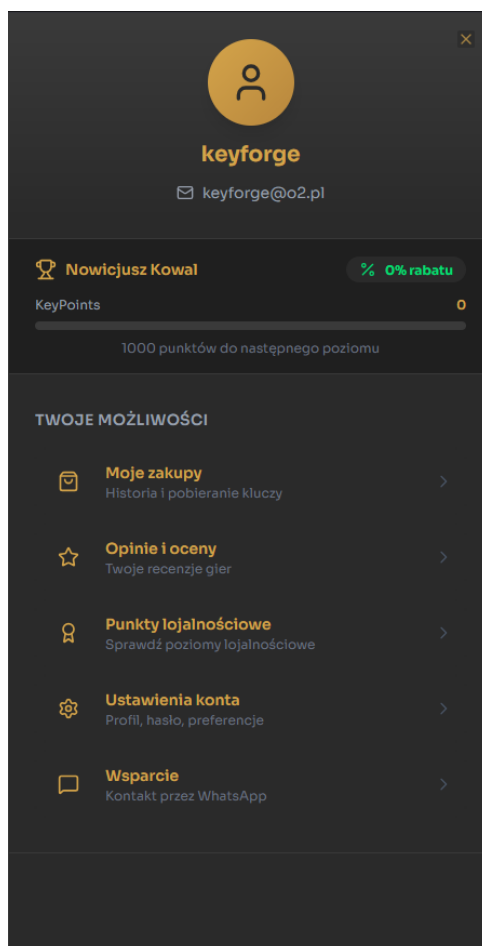
Rysunek 16: Widok koszyka z podsumowaniem zamówienia i statusem punktów lojalnościowych.

Główne elementy interfejsu:

- **Lista produktów:** Dynamiczne zestawienie pozycji dodanych do zamówienia. Każdy element zawiera miniaturę, nazwę, przypisaną platformę oraz interaktywne kontrolki ilości (inkrementacja/dekrementacja) komunikujące się z `product-service`.
- **System KeyPoints:** Zintegrowany z `user-service` moduł lojalnościowy. Użytkownik widzi prognozowaną liczbę punktów do zdobycia za bieżące zakupy oraz pasek postępu wskazujący drogę do kolejnego poziomu (np. „Uczeń Kuźni”). System ten zachęca do retencji poprzez wizualizację progresu konta.
- **Podsumowanie:** Sekcja szczegółowego wyliczenia kwoty do zapłaty. Algorytm uwzględnia rabaty produktowe oraz potencjalne rabaty lojalnościowe, prezentując sumę końcową przed przekierowaniem do bramki płatniczej.
- **Integracja z płatnościami:** Przycisk „Przejdź do płatności” inicjuje proces w mikroservisie `payment-service`, przygotowując sesję Stripe w oparciu o unikalny identyfikator zamówienia.

Panel profilu użytkownika

Interfejs użytkownika został wzbogacony o dynamiczny panel boczny (Rysunek 17), wywoływany po kliknięciu w nazwę profilu w nagłówku strony. Rozwiązanie to pozwala na szybki dostęp do zarządzania kontem bez konieczności przeładowywania bieżącego widoku, co wpisuje się w standardy **Single Page Application**.



Rysunek 17: Boczny panel nawigacyjny zalogowanego użytkownika.

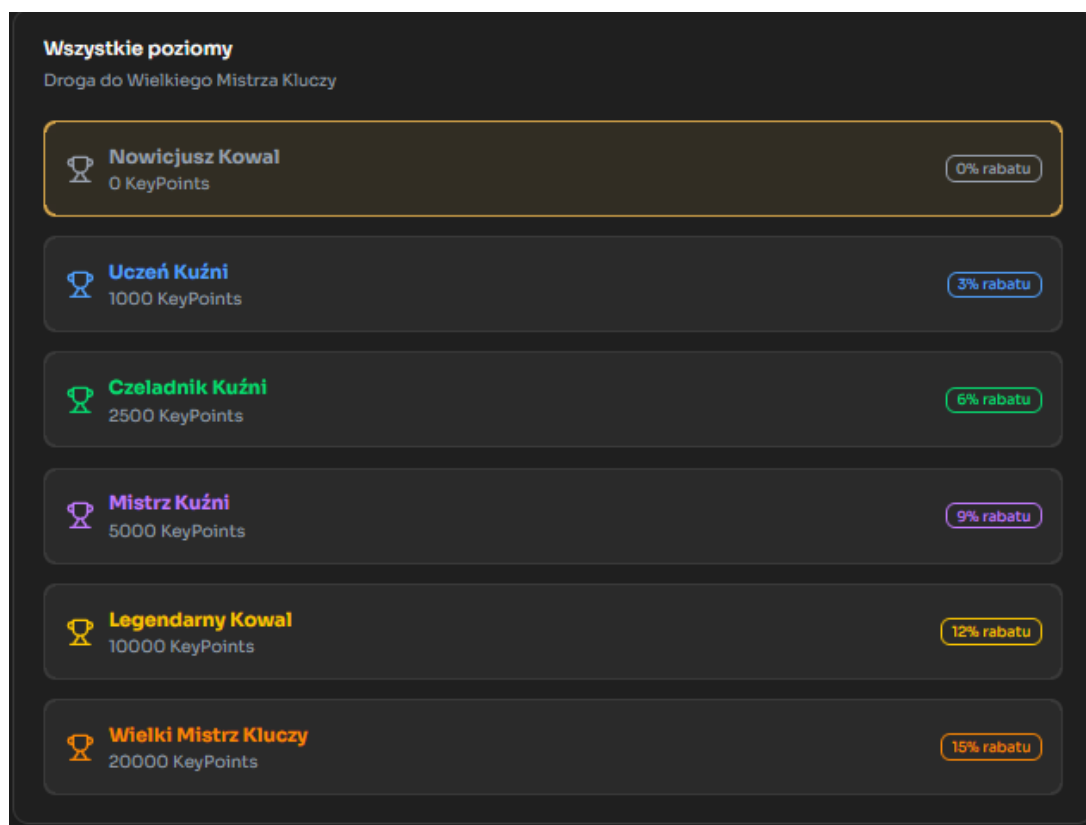
Główne elementy interfejsu:

- **Nagłówek profilu:** Prezentuje nazwę użytkownika oraz adres e-mail powiązany z kontem, pobierane z **user-service** po pomyślnej autoryzacji w usłudze Keycloak.
- **Status Lojalnościowy:** Wizualna reprezentacja aktualnego poziomu (np. „Nowicjusz Kowal”) wraz z paskiem postępu punktów **KeyPoints**. Moduł ten informuje również o aktualnie przysługującym rabacie lojalnościowym (np. 3%), co jest wyliczane w czasie rzeczywistym.
- **Menu szybkich akcji:** Zbiór odnośników do kluczowych sekcji:
 - *Moje zakupy:* Historia transakcji i dostęp do zakupionych kluczy cyfrowych.
 - *Opinie i oceny:* Zarządzanie wystawionymi recenzjami produktów.
 - *Punkty lojalnościowe:* Wgląd na możliwe do osiągnięcia poziomy lojalnościowe oraz aktualny poziom i postęp użytkownika do następnego tytułu.

- *Ustawienia konta*: Zarządzanie preferencjami i danymi profilowymi.
- *Wsparcie*: Bezpośredni odnośnik do kanału pomocy technicznej (WhatsApp).

System Poziomów Lojalnościowych (KeyPoints)

System KeyForge implementuje zaawansowany program lojalnościowy oparty na punktach **KeyPoints**, które użytkownicy zdobywają za każdą sfinalizowaną transakcję. **Za każdą wydaną złotówkę, użytkownik otrzymuje 2 KeyPoints**. Mechanika ta, przedstawiona na Rysunku 18, ma na celu zwiększenie retencji klientów poprzez oferowanie progresywnych korzyści finansowych.



Rysunek 18: Zestawienie progów punktowych i stałych rabatów w systemie lojalnościowym.

Struktura poziomów i korzyści: System definiuje sześć unikalnych rang, z których każda przypisuje do konta użytkownika stały, procentowy rabat na cały asortyment:

Nazwa Poziomu	Próg KeyPoints	Staly Rabat
Nowicjusz Kowal	0 pkt	0%
Uczeń Kuźni	1 000 pkt	3%
Czeladnik Kuźni	2 500 pkt	6%
Mistrz Kuźni	5 000 pkt	9%
Legendarny Kowal	10 000 pkt	12%
Wielki Mistrz Kluczy	20 000 pkt	15%

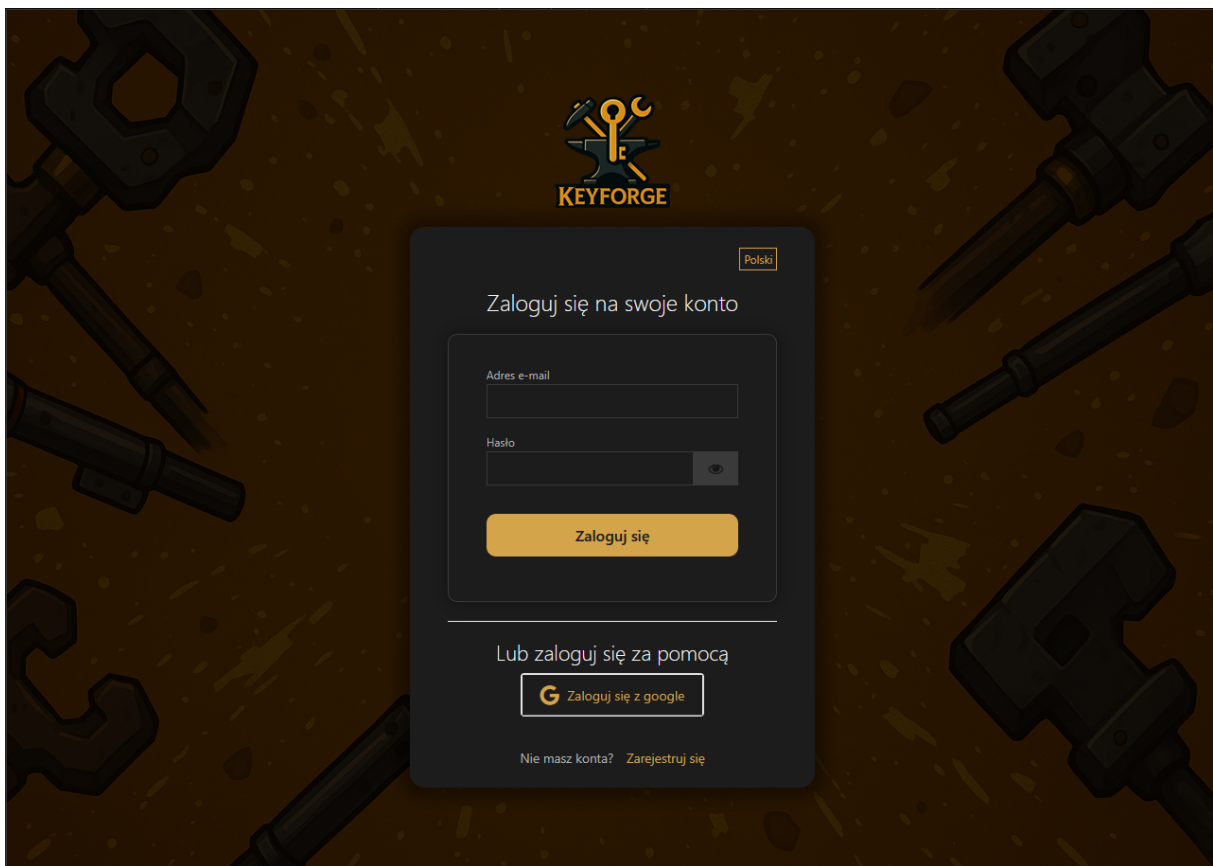
Tabela 2: Specyfikacja progów lojalnościowych i zniżek.

Główne elementy interfejsu:

- **Przelicznik punktowy:** Punkty są naliczane przez `user-service` po otrzymaniu sygnału o odbiorze zamówienia z mikroservisu `order-service`.
- **Dynamiczne rabatowanie:** Aktualny poziom lojalnościowy użytkownika jest weryfikowany podczas kalkulacji sumy zamówienia w `order-service`, co pozwala na automatyczne odjęcie odpowiedniego procentu od kwoty końcowej.
- **Warstwa prezentacji:** Interfejs wykorzystuje kolory sygnalizacyjne (od szarości nowicjusza po złoty kolor Wielkiego Mistrza), aby wizualnie podkreślić prestiż posiadanego statusu.

Panel logowania i autoryzacji

Dostęp do funkcji personalizowanych, takich jak system **KeyPoints** czy historia zakupów, realizowany jest poprzez dedykowany moduł logowania (Rysunek 19). Logika backendowa opiera się na protokole **OAuth2** i **OpenID Connect** obsługiwanym przez serwer `keycloak`.



Rysunek 19: Interfejs logowania do platformy KeyForge z integracją OAuth2.

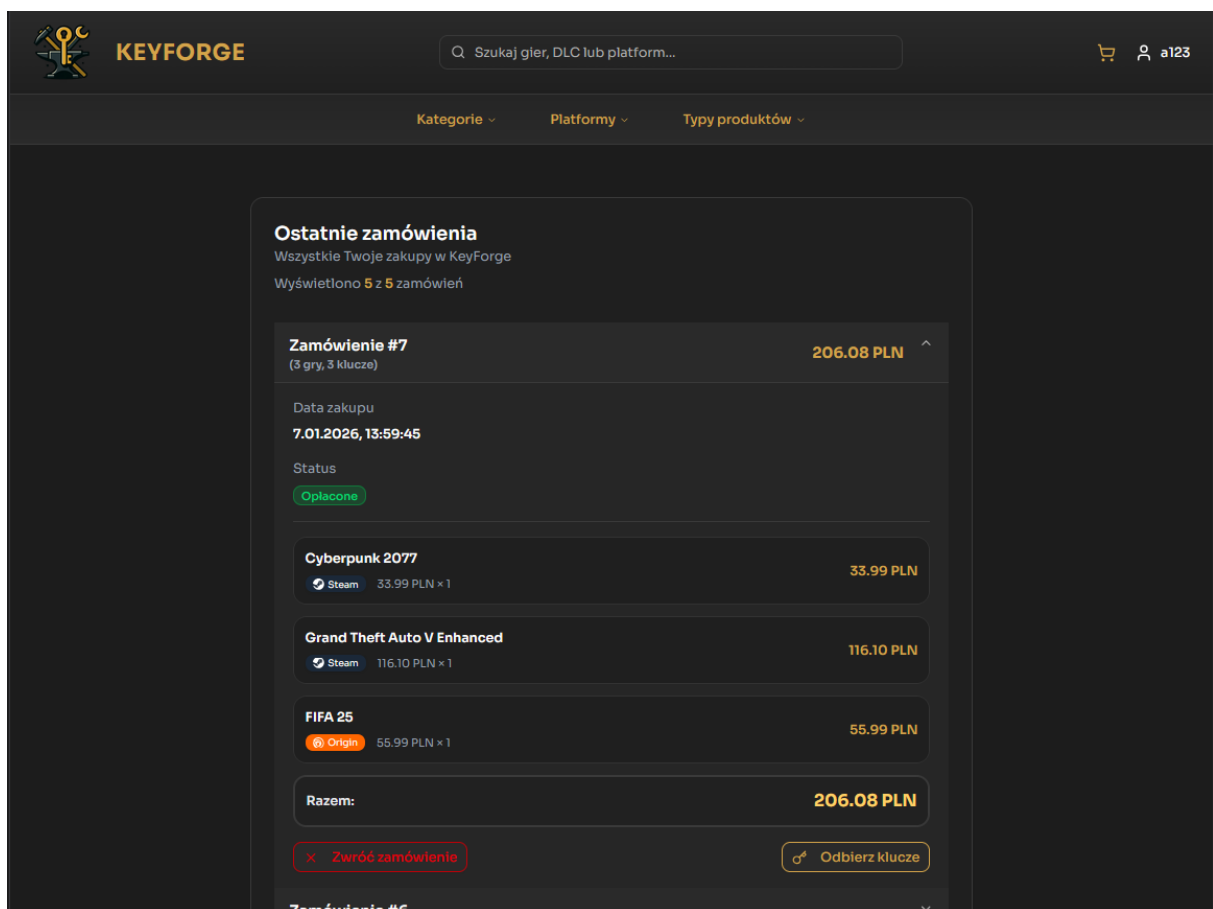
Główne elementy modułu autoryzacji:

- **Uwierzytelnianie hybrydowe:** System umożliwia logowanie za pomocą tradycyjnego formularza (e-mail i hasło) lub poprzez integrację z dostawcą zewnętrznym (**Google OAuth2**), co znacząco skraca proces *onboardingu* użytkownika.

- **Zarządzanie sesją:** Po pomyślnym zalogowaniu, **keycloak** generuje tokeny **JWT** (Access Token i Refresh Token), które są wykorzystywane przez **api-gateway** do autoryzacji żądań kierowanych do poszczególnych mikroservisów.
- **Obsługa lokalizacji:** Interfejs wspiera wybór języka (widoczny selektor „Polski”), co pozwala na dynamiczne dostosowanie komunikatów błędu i etykiet do preferencji użytkownika.

Historia zamówień i odbiór kluczy

Sekcja „Ostatnie zamówienia” (Rysunek 20) stanowi interfejs końcowy procesu zakupowego, zintegrowany z **order-service** oraz **user-service**. Widok ten zarządza cyklem życia zamówienia po dokonaniu płatności, oferując użytkownikowi wybór między sfinalizowaniem transakcji a zwrotem środków.



Rysunek 20: Widok szczegółowy opłaconego zamówienia z opcjami odbioru i zwrotu.

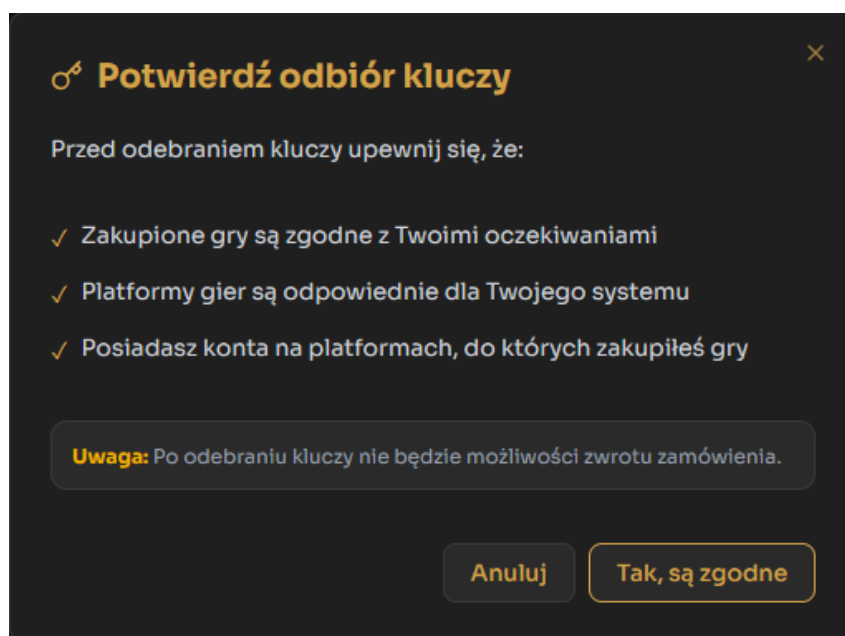
Mechanika i logika biznesowa widoku:

- **Status zamówienia:** Każda transakcja posiada czytelną etykietę statusu (np. „Opłacone”), co informuje użytkownika o poprawnym przetworzeniu płatności przez mikroservis Stripe.
- **Odbiór kluczy cyfrowych:** Przycisk „Odbierz klucze” otworzy dialog widoczny na Rysunku 21. Potwierdzenie odbioru jest procesem nieodwracalnym. Wyświetlenie

kluczy do gier (np. Cyberpunk 2077, FIFA 25) jest tożsame z finalizacją zakupu, co zgodnie z polityką treści cyfrowych blokuje możliwość dokonania zwrotu.

- **Naliczanie punktów KeyPoints:** Dopiero po potwierdzeniu odbioru kluczy, system automatycznie aktualizuje saldo punktowe użytkownika w `user-service`, przydzielając punkty lojalnościowe widoczne wcześniej w podsumowaniu koszyka.
- **Prawo do odstąpienia:** Dopóki klucze nie zostaną wyświetlone, użytkownik ma dostęp do funkcji „Zwróć zamówienie”. Pozwala to na automatyczną inicjację procedury refundacji środków za pośrednictwem `payment-service`.
- **Szczegóły pozycji:** Rozwijane menu zamówienia prezentuje kompletne zestawienie: nazwy gier, przypisane platformy (Steam, Origin), ceny jednostkowe oraz całkowitą kwotę transakcji.

Przed ostatecznym wyświetleniem zakupionych kodów, system wywołuje modalne okno dialogowe „Potwierdź odbiór kluczy” (Rysunek 21). Jest to krytyczny punkt styku z użytkownikiem, pełniący funkcję prawną i informacyjną.



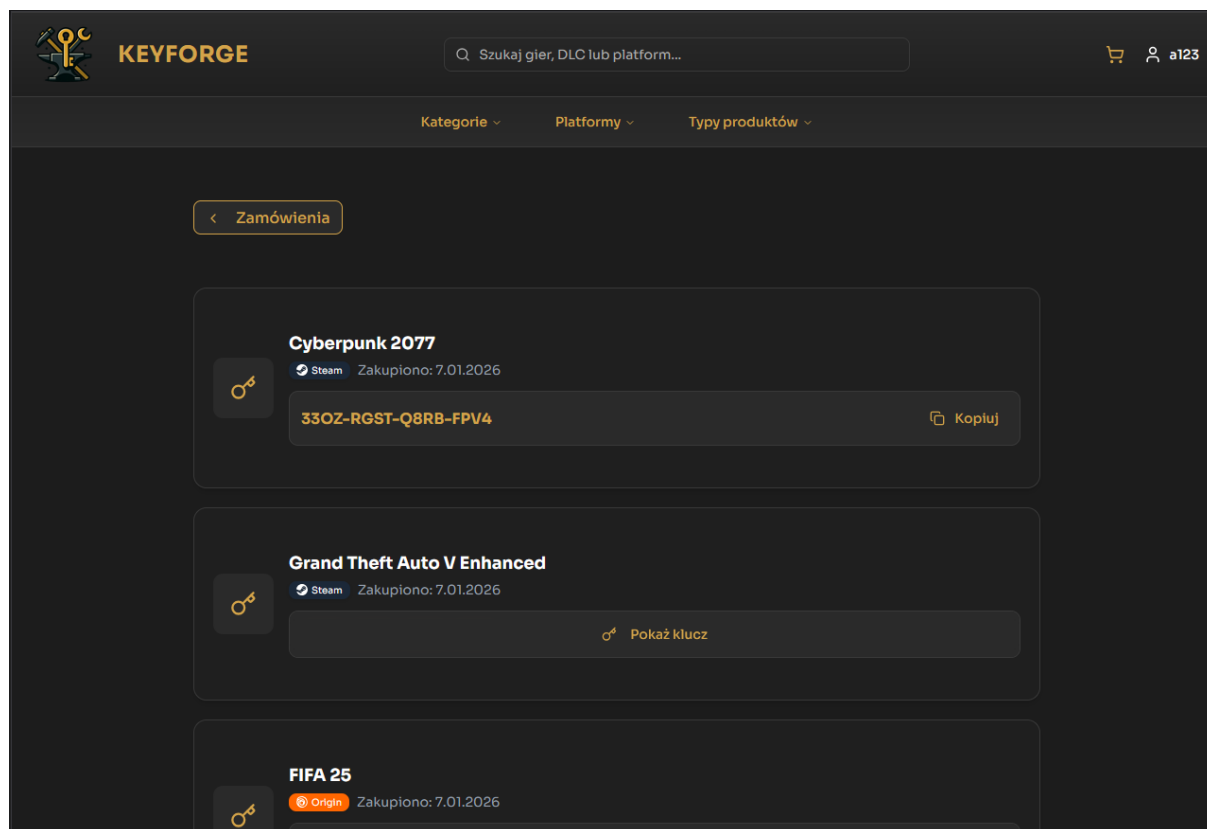
Rysunek 21: Okno walidacji przed ostatecznym wydaniem kluczy cyfrowych.

Funkcje komunikatu:

- **Walidacja techniczna:** System wymusza na użytkowniku ponowną weryfikację zgodności platformy (np. Steam vs Origin) oraz wymagań sprzętowych, co minimalizuje liczbę reklamacji wynikających z pomyłek klienta.
- **Zastrzeżenie prawne:** Wyraźne ostrzeżenie o utracie prawa do zwrotu po odsłonięciu klucza zabezpiecza interesy platformy zgodnie z dyrektywami o dostarczaniu treści cyfrowych.
- **Dwustopniowe potwierdzenie:** Przycisk „Tak, są zgodne” stanowi ostateczną zgodę na sfinalizowanie transakcji, po której następuje asynchroniczna aktualizacja bazy danych i odblokowanie widoku kluczy.

Widok odebranych kluczy cyfrowych

Po sfinalizowaniu procesu odbioru i akceptacji warunków w oknie modalnym, użytkownik uzyskuje dostęp do docelowej zawartości zamówienia (Rysunek 22). W tym momencie `order-service` zmienia status pozycji na „Odebrane”, a interfejs prezentuje unikalne kody aktywacyjne.



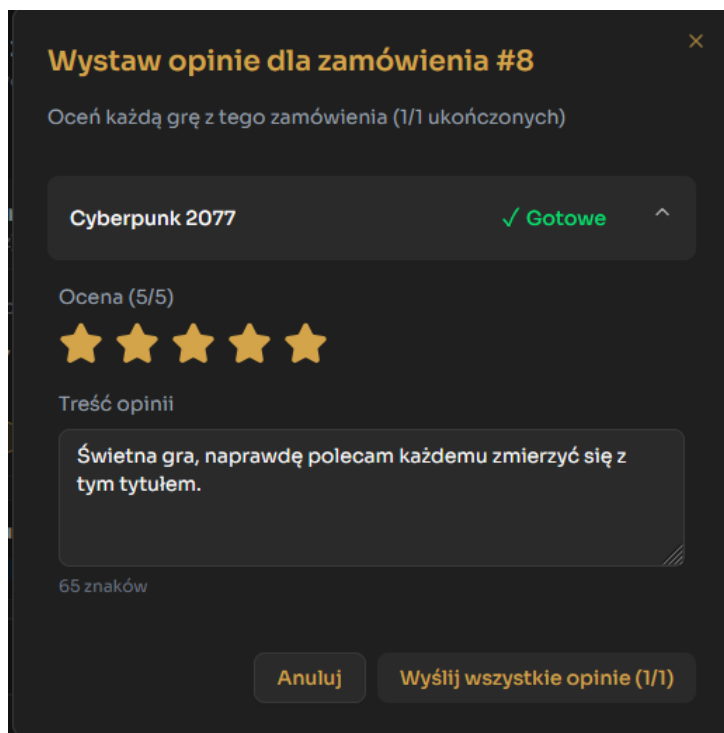
Rysunek 22: Interfejs prezentujący zakupione klucze cyfrowe z opcją kopiowania.

Główne elementy interfejsu:

- **Prezentacja kodów:** Każdy produkt wyświetlany jest jako osobna karta zawierająca nazwę gry, platformę (np. Steam, Origin) oraz datę zakupu.
- **Funkcja szybkiego kopiowania:** Przy każdym jawnym kluczu znajduje się przycisk „Kopiuj”, który automatycznie przenosi kod do schowka systemowego, co znacząco podnosi User Experience podczas aktywacji gry na zewnętrznych platformach.
- **Maskowanie zawartości:** System domyślnie maskuje klucze, których użytkownik jeszcze nie odsłonił (przycisk „Pokaż klucz”), co pozwala na selektywne zarządzanie zakupionymi produktami i dodatkowo zabezpiecza przed przypadkowym podejrzeniem kodu przez osoby trzecie.
- **Blokada zwrotu:** Zgodnie z wcześniejszym komunikatem, wygenerowanie tego widoku trwale usuwa opcję „Zwróć zamówienie” dla danej transakcji w bazie danych, kończąc cykl życia zamówienia jako w pełni zrealizowanego.

System wystawiania opinii i ocen

Po sfinalizowaniu odbioru kluczy, system odblokowuje moduł oceniania produktów (Rysunek 23). Funkcjonalność ta jest obsługiwana przez `review-service`.



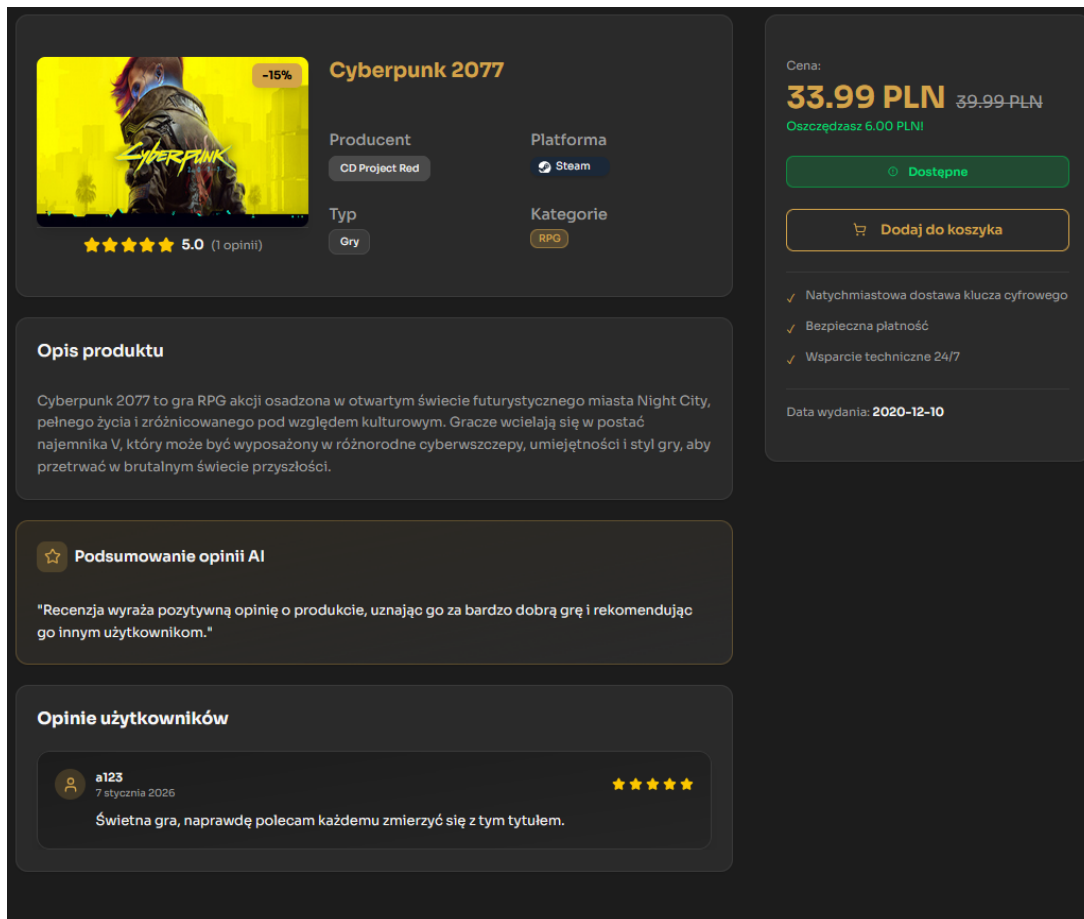
Rysunek 23: Interfejs wystawiania opinii dla produktów z zamówienia.

Charakterystyka procesu oceniania:

- **Weryfikacja zakupu:** Możliwość wystawienia opinii jest ściśle powiązana z statusem zamówienia „Odebrane”, co gwarantuje, że wszystkie recenzje w systemie pochodzą od faktycznych nabywców gier.
- **Skala ocen i treść:** Użytkownik przypisuje produktowi ocenę w skali 1-5 gwiazdek oraz opcjonalną recenzję tekstową. System w czasie rzeczywistym monitoruje liczbę znaków wprowadzonych przez klienta.
- **Moderacja AI:** Przesłana treść jest przekazywana do `ai-service`, który analizuje opinię pod kątem wulgaryzmów oraz mowy nienawiści, przypisując jej status REJECTED lub APPROVED.
- **Status ukończenia:** W przypadku zamówień wieloprezedmiotowych, modal wyświetla postęp (np. „1/1 ukończonych”), co ułatwia zarządzanie procesem oceniania całej zawartości koszyka.

Integracja z AI i dynamiczne streszczenia recenzji

System KeyForge wykorzystuje zaawansowany moduł **ai-service** do analizy i agregacji opinii użytkowników. Proces ten wykracza poza standardową moderację, oferując dynamiczne podsumowania treści (Rysunek 24), co znacząco ułatwia potencjalnym nabywcom ocenę jakości produktu.



Rysunek 24: Karta produktu z widocznym podsumowaniem wygenerowanym przez AI oraz zatwierdzoną opinią.

Przebieg procesu i wpływ na interfejs:

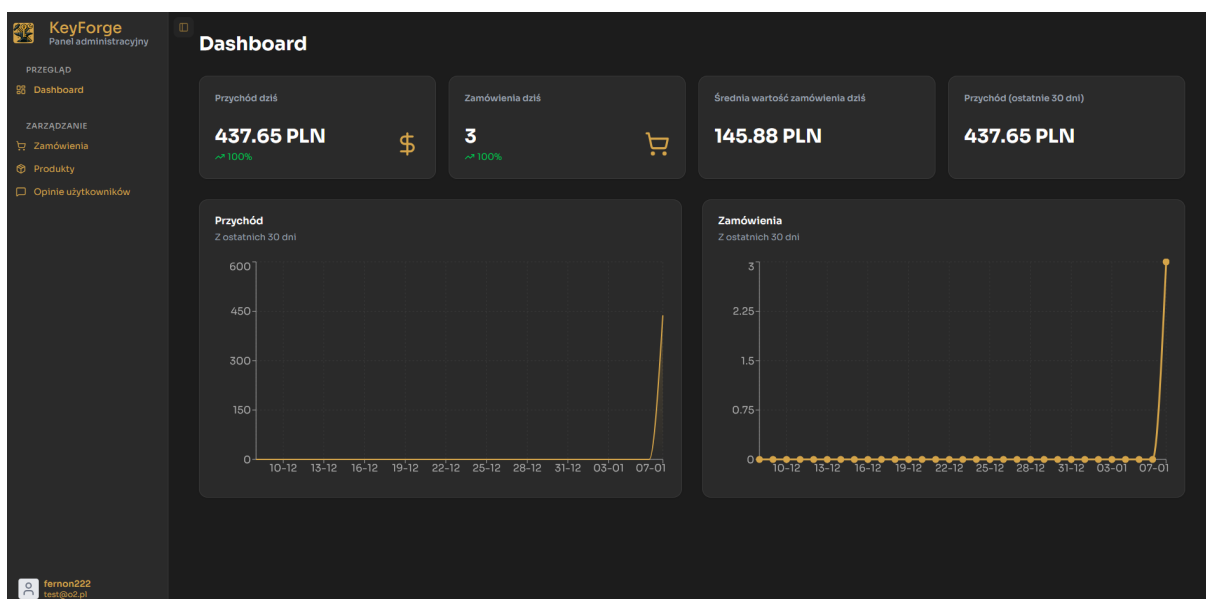
- **Automatyczna moderacja:** Każda nowa opinia po przesłaniu trafia do kolejki mikros serwisu AI, który weryfikuje ją pod kątem zgodności z regulaminem. Po uzyskaniu statusu **APPROVED**, recenzja staje się natychmiast widoczna w sekcji „Opinie użytkowników”.
- **Generowanie streszczeń (AI Summary):** System analizuje wszystkie zatwierdzone recenzje dla danego produktu i generuje zwięzłe podsumowanie w sekcji „Podsumowanie opinii AI”. Funkcjonalność ta wyciąga kluczowe argumenty (pozytywne i negatywne), prezentując je w formie obiektywnego werdyktu.
- **Skalowanie społecznościowe:** Wraz ze wzrostem liczby recenzji, podsumowanie AI staje się coraz bardziej precyzyjne, co pozwala uniknąć konieczności czytania setek indywidualnych komentarzy przez klienta.

Panel administratora

Panel administratora stanowi dedykowany moduł systemu, przeznaczony do nadzoru nad procesami biznesowymi oraz zarządzania bazą danych platformy. Dostęp do tej sekcji realizowany jest poprzez dedykowany punkt wejścia `/admin`.

Dashboard i Bezpieczeństwo Panelu

Bezpieczeństwo panelu administracyjnego (Rysunek 25) opiera się na wielowarstwowej kontroli dostępu, integrującej infrastrukturę serwerową z logiką nawigacyjną aplikacji klienckiej. Centralnym punktem autoryzacji jest usługa **Keycloak**, która wystawia tokeny JWT zawierające role przypisane do użytkownika.



Rysunek 25: Główny pulpit nawigacyjny (Dashboard) panelu administracyjnego.

Mechanizmy kontroli dostępu i routing:

- **Weryfikacja ról przez TanStack Router:** System wykorzystuje bibliotekę **TanStack Router** do zarządzania procesem routingu. Przed wyrenderowaniem widoku administracyjnego, router wykonuje funkcję `beforeLoad`, która sprawdza obecność wymaganej roli w profilu użytkownika. W przypadku braku uprawnień, użytkownik jest automatycznie przekierowywany na stronę logowania lub stronę główną, co zapobiega dostępowi do kodu komponentów administracyjnych.
- **Zabezpieczenie na poziomie API Gateway:** Niezależnie od walidacji frontendowej, `api-gateway` weryfikuje każde przychodzące żądanie do mikroservisów pod kątem ważności tokena i posiadanych uprawnień. Zapewnia to integralność danych nawet w przypadku próby ręcznej manipulacji adresem URL.
- **Dynamiczny Dashboard:** Widok prezentuje kluczowe wskaźniki KPI (przychodów i wolumenu zamówień) pobierane z `order-service`.
- **Personalizacja sesji:** Tożsamość zalogowanego administratora jest stale wyświetlana w dolnej części panelu bocznego, co ułatwia audyt sesji i potwierdza poprawność kontekstu autoryzacyjnego.

Zarządzanie zamówieniami

Moduł zarządzania zamówieniami (Rysunek 26) stanowi centralny punkt kontroli operacyjnej systemu. Widok ten agreguje dane z mikroservisu `order-service` oraz `payment-service`, umożliwiając administratorom pełny wgląd w historię transakcji oraz bieżące procesy zakupowe użytkowników.

ID	User ID	Produkty	Cena	Status	Data	Akcje
#8	be489041-e1a2-4c1d-a9ab-d828756d9a6c	1 (1 szt.)	33.99 PLN	Odebrane	7.01.2026	🔍
#7	be489041-e1a2-4c1d-a9ab-d828756d9a6c	3 (3 szt.)	206.08 PLN	Odebrane	7.01.2026	🔍
#6	be489041-e1a2-4c1d-a9ab-d828756d9a6c	3 (3 szt.)	197.58 PLN	Oczekiwanie na płatność	7.01.2026	🔍
#5	be489041-e1a2-4c1d-a9ab-d828756d9a6c	3 (3 szt.)	197.58 PLN	Opłacone	7.01.2026	🔍
#4	d7ee821a-53f3-4931-adbc-3f96013a0e0a	4 (4 szt.)	207.46 PLN	Odebrane	20.11.2025	🔍
#3	d7ee821a-53f3-4931-adbc-3f96013a0e0a	4 (4 szt.)	207.46 PLN	Opłacone	20.11.2025	🔍
#2	be489041-e1a2-4c1d-a9ab-d828756d9a6c	3 (3 szt.)	137.47 PLN	Oczekiwanie na płatność	20.11.2025	🔍
#1	be489041-e1a2-4c1d-a9ab-d828756d9a6c	3 (3 szt.)	137.47 PLN	Odebrane	20.11.2025	🔍

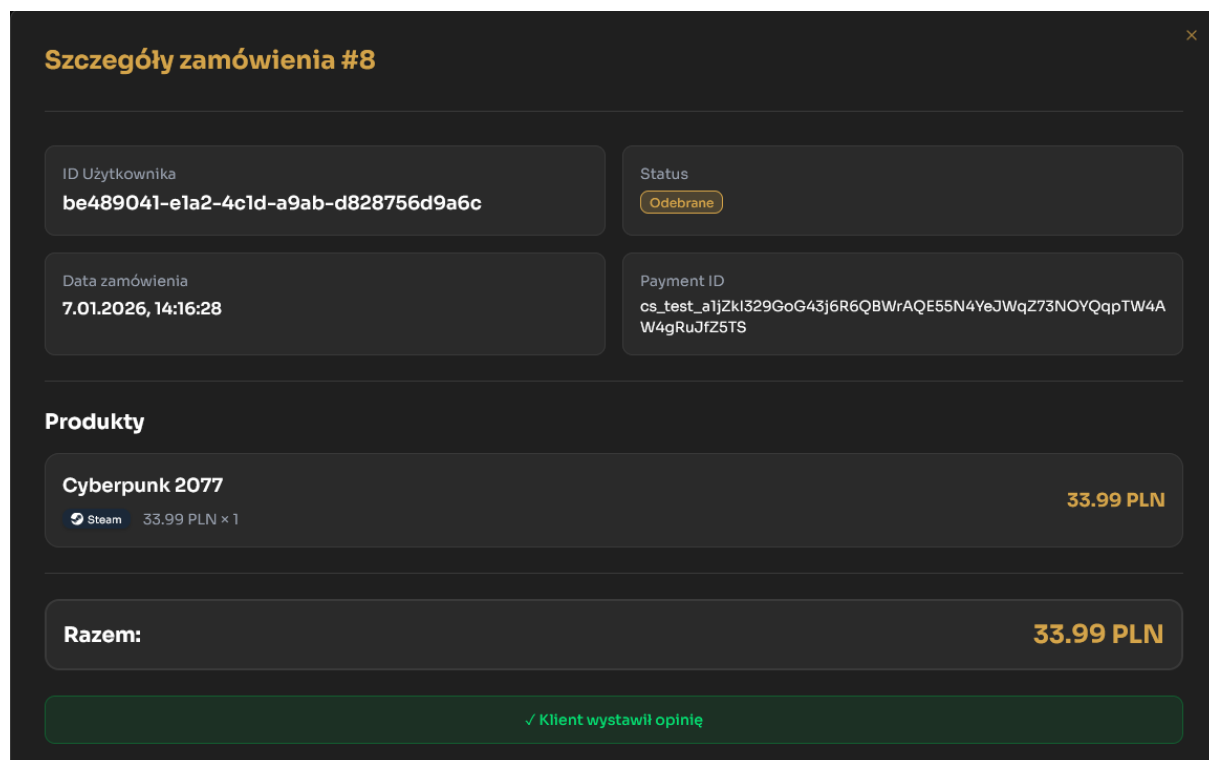
Rysunek 26: Panel administracyjny - lista zamówień klientów.

Funkcjonalności i mechanizmy systemowe:

- **Zaawansowany system filtrowania:** Interfejs oferuje dynamiczne wyszukiwanie zamówień po identyfikatorze (ID zamówienia), unikalnym ID użytkownika (UUID z Keycloak) oraz statusie transakcji. Umożliwia to kadrze zarządzającej szybką lokalizację konkretnych operacji finansowych.
- **Zarządzanie stanem zamówienia:** Każdy rekord prezentuje aktualny status (np. *Odebrane*, *Opłacone*, *Oczekiwanie na płatność*). Administratorzy mają możliwość ręcznej zmiany statusu za pomocą listy rozwijanej, co jest kluczowe w procesach obsługi reklamacji lub manualnej weryfikacji płatności.
- **Paginacja i sortowanie:** Implementacja po stronie frontendu wspiera efektywne przeglądanie dużych zbiorów danych. Użytkownik może definiować liczbę rekordów na stronę oraz sortować listę według daty utworzenia (rosnąco/malejąco).
- **Szczegółowa analityka wiersza:** Tabela wyświetla zagregowane informacje, takie jak liczba zakupionych sztuk produktów oraz całkowita wartość zamówienia. Ikona podglądu (akcja) pozwala na przejście do widoku szczegółowego danej transakcji.

Szczegóły zamówienia klienta

Administrator posiada możliwość wglądu w szczegółowe dane konkretnej transakcji (Rysunek 27), co jest niezbędne do obsługi procesów wsparcia technicznego oraz weryfikacji poprawności przepływu płatności. Widok ten agreguje informacje z trzech systemów: `order-service`, `payment-service`.



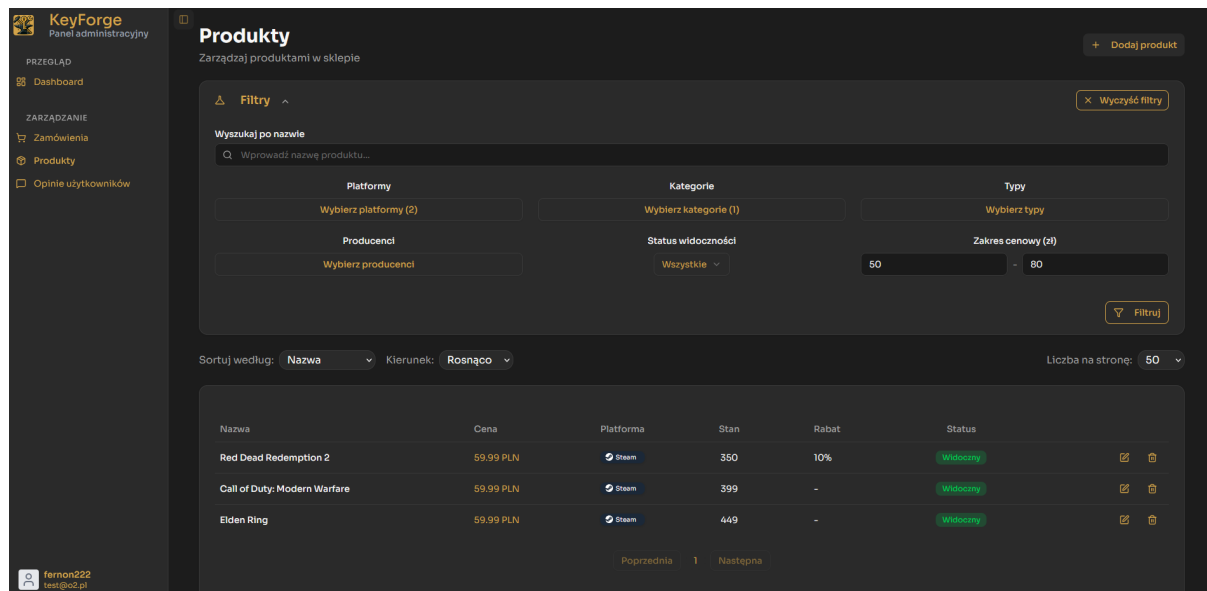
Rysunek 27: Szczegółowy podgląd zamówienia nr 8 w panelu administracyjnym.

Funkcjonalność podglądu zamówienia:

- **Dane identyfikacyjne:** Panel wyświetla unikalny identyfikator użytkownika z systemu Keycloak oraz dokładną datę i godzinę złożenia zamówienia.
- **Weryfikacja płatności:** Wyświetlany jest pełny *Payment ID* (pochodzący z Stripe API), co umożliwia administratorowi szybkie odnalezienie transakcji bezpośrednio w panelu Stripe w przypadku wystąpienia sporów płatniczych.
- **Wykaz produktów:** Sekcja prezentuje listę gier wchodzących w skład zamówienia wraz z ich cenami w momencie zakupu oraz przypisanymi platformami.
- **Status interakcji klienta:** System informuje administratora o aktywności użytkownika po zakupie. Przykładowo, zielona etykieta „Klient wystawił opinię” sygnalizuje, że proces zakupu i oceny został w pełni zakończony, a rekord w `review-service` jest powiązany z tym zamówieniem.
- **Podsumowanie:** Wyraźnie zaznaczona kwota łączna zamówienia pozwala na szybką walidację poprawności naliczenia sumy końcowej.

Zarządzanie asortymentem produktów

Moduł zarządzania produktami (Rysunek 28) umożliwia administratorom pełną kontrolę nad katalogiem gier. Interfejs integruje się z `product-service`, oferując zaawansowane mechanizmy wyszukiwania oraz zarządzania stanami magazynowymi i widocznością ofert.



Rysunek 28: Panel administracyjny – lista produktów z rozwiniętym modulem filtrów.

Zarządzanie produktami w panelu administracyjnym:

- **Zaawansowany system filtrowania:** Panel umożliwia precyzyjne zawężanie listy produktów poprzez parametry takie jak: nazwa (wyszukiwanie tekstowe), platformy, kategorie, producenci oraz status widoczności. Dodatkowo wprowadzono filtr zakresu cenowego, co pozwala na szybki audyt polityki cenowej.
- **Tabela produktów:** Każdy wiersz prezentuje kluczowe informacje o produkcie: stan magazynowy (liczba dostępnych kluczy), aktualny rabat oraz status „Widoczny/Ukryty”.
- **Akcje CRUD:** Interfejs udostępnia bezpośrednie skróty do edycji istniejących pozycji oraz przycisk „Dodaj produkt”, który inicjuje proces wprowadzania nowej gry do bazy danych. Funkcja usuwania jest zabezpieczona dodatkowym potwierdzeniem, aby zapobiec przypadkowej utracie danych.
- **Dynamiczne sortowanie:** Administrator może sortować asortyment według dowolnej kolumny (np. po cenie lub nazwie), co w połączeniu z paginacją pozwala na efektywne zarządzanie bazą PostgreSQL.
- **Integracja z routingiem:** Podobnie jak w pozostałych sekcjach panelu, dostęp do tej zakładki jest chroniony przez autoryzację **TanStack Router** w oparciu o role pobrane z serwera Keycloak.

Zarządzanie detalami produktu (CRUD)

Interfejs edycji i tworzenia produktu (Rysunek 29) to rozbudowany moduł formularzowy, zaprojektowany w formie okna modalnego z podziałem na sekcje tematyczne (*Tabs*). Pozwala on na precyzyjne definiowanie danych produktu, które są następnie dystrybuowane do bazy PostgreSQL mikroservisu `product-service`.

Edytuj produkt
Wprowadź zmiany w szczegółach produktu

Informacje główne Opis Kategorie

Podstawowe informacje

Nazwa produktu
Cyberpunk 2077

Data wydania ID logo
10.12.2020 1091500

Ceny i dostępność

Cena (PLN) Rabat (%) Stan magazynowy
39,99 15 294

Cena po rabacie: **33.99 PLN**

Klasyfikacja

Anuluj Zapisz zmiany

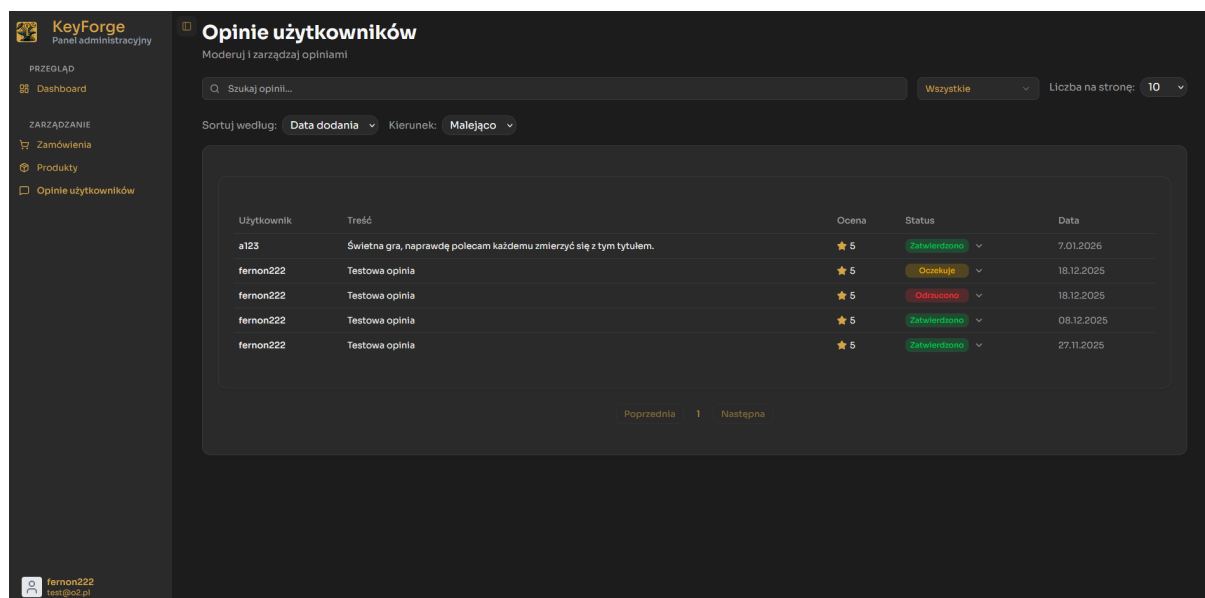
Rysunek 29: Modalny interfejs edycji danych produktu Cyberpunk 2077.

Struktura i funkcjonalności formularza:

- **Informacje główne:** Administrator definiuje podstawowe atrybuty gry, takie jak nazwa, data wydania oraz logotyp. Dane te stanowią fundament widoku karty produktu w panelu klienta.
- **Ceny i dostępność:** Sekcja umożliwia zarządzanie polityką cenową. System automatycznie przelicza cenę końcową na podstawie wprowadzonej ceny bazowej oraz procentowej wartości rabatu. Administrator ma również wgląd w aktualny stan magazynowy (liczbę dostępnych kluczy).
- **Klasyfikacja i Opis:** Osobne zakładki pozwalają na przypisywanie gier do odpowiednich kategorii i platform oraz edycję bogatych opisów marketingowych.
- **Integracja z AI:** W ramach edycji opisu, administrator posiada funkcję wymuszenia ponownego wygenerowania podsumowania AI. Pozwala to na odświeżenie treści podsumowania recenzji lub stworzenia nowego, w przypadku wystąpienia błędów.
- **Logika stanowa:** Formularz tworzenia nowego obiektu wykorzystuje tę samą strukturę komponentów, co widok edycji, inicjując puste pola wejściowe. Walidacja danych odbywa się w czasie rzeczywistym przed wysłaniem żądania API.

Moderação i zarządzanie opiniami

Ostatnim modulem panelu administracyjnego jest system nadzoru nad recenzjami użytkowników (Rysunek 30). Sekcja ta, obsługiwana przez `review-service`, pozwala administratorom na ostateczną weryfikację treści.



Rysunek 30: Widok panelu moderacji opinii z widocznymi statusami weryfikacji.

Komponenty systemu moderacji:

- **Centralny rejestr recenzji:** Tabela prezentuje chronologiczną listę wszystkich wystawionych opinii. Administrator ma wgląd w tożsamość autora, treść recenzji, przyznaną ocenę punktową (gwiazdki) oraz datę publikacji.
- **Statusy moderacji:** Każda opinia posiada przypisany jeden z trzech stanów:
 - **Zatwierdzono** (zielony): Treść przeszła pozytywnie weryfikację AI lub manualną i jest widoczna dla wszystkich użytkowników.
 - **Oczekuje** (żółty): Opinia wymaga ręcznego sprawdzenia przez administratora (np. w przypadku niepewności algorytmu AI).
 - **Odrzucono** (czerwony): Treść naruszająca regulamin, która została trwale ukryta w widoku publicznym.
- **Interaktywna zmiana stanu:** Administratorzy mogą manualnie nadpisać decyzję podjętą przez `ai-service` za pomocą listy rozwijanej w kolumnie „Status”. Każda zmiana statusu, z wyjątkiem Odrzucono->Oczekuje, wymusza automatyczną aktualizację podsumowania AI na karcie danej gry.
- **Narzędzia filtrowania i sortowania:** Podobnie jak w module zamówień, system wspiera zaawansowane wyszukiwanie tekstowe oraz filtrowanie po statusach, co pozwala na szybkie odnalezienie nowych opinii wymagających zatwierdzenia.
- **Skalowalność i wydajność:** Wykorzystanie bazy MongoDB w mikroserwisie opinii umożliwia płynne zarządzanie dużą liczbą rekordów tekstowych, zachowując wysoką responsywność interfejsu administracyjnego nawet przy intensywnym ruchu.

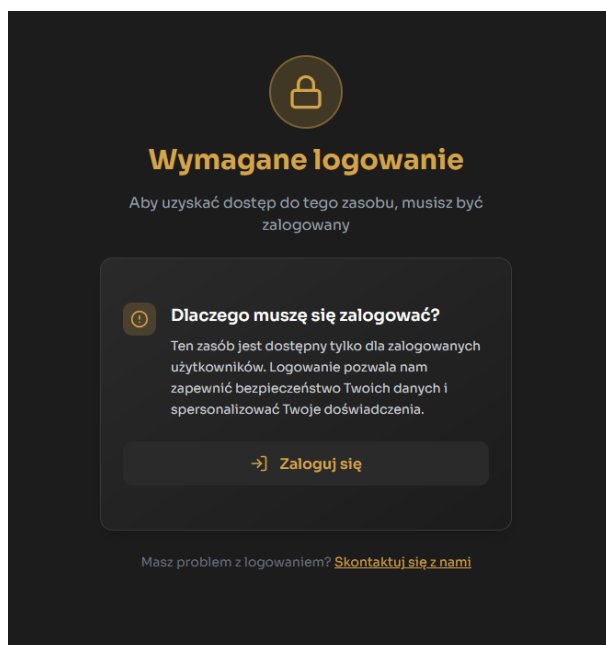
Autoryzacja i obsługa błędów dostępu

System KeyForge implementuje wielopoziomowy model autoryzacji, który dynamicznie dostosowuje widoczność komponentów interfejsu do uprawnień zalogowanego użytkownika. Wykorzystanie biblioteki **TanStack Router** pozwala na przechwytywanie prób nieautoryzowanego dostępu jeszcze przed załadowaniem docelowych widoków.

Mechanizmy walidacji uprawnień

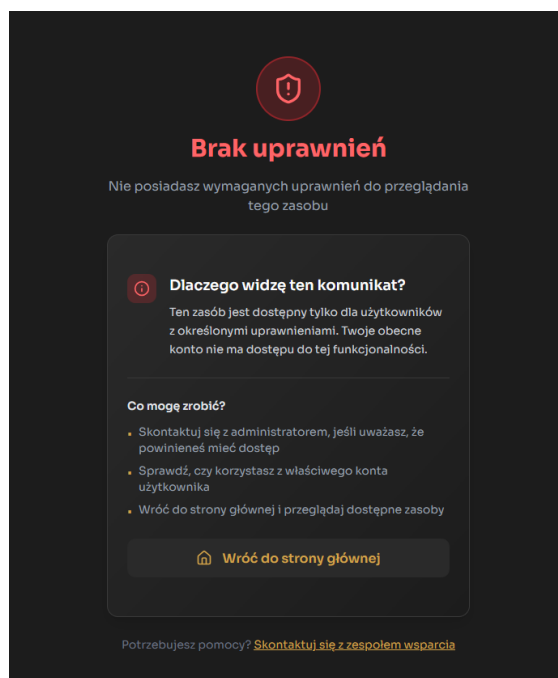
Wyróżnia się trzy kluczowe scenariusze obsługi błędów dostępu, które prezentowane są użytkownikowi w formie dedykowanych ekranów informacyjnych:

Brak zalogowania (Unauthenticated): W przypadku próby dostępu do zasobów prywatnych (np. koszyka, historii zamówień) bez aktywnej sesji, system wyświetla ekran blokady (Rysunek 31). Jasny komunikat wyjaśnia konieczność personalizacji doświadczenia i bezpieczeństwa danych, oferując bezpośredni przycisk przekierowujący do formularza logowania Keycloak.



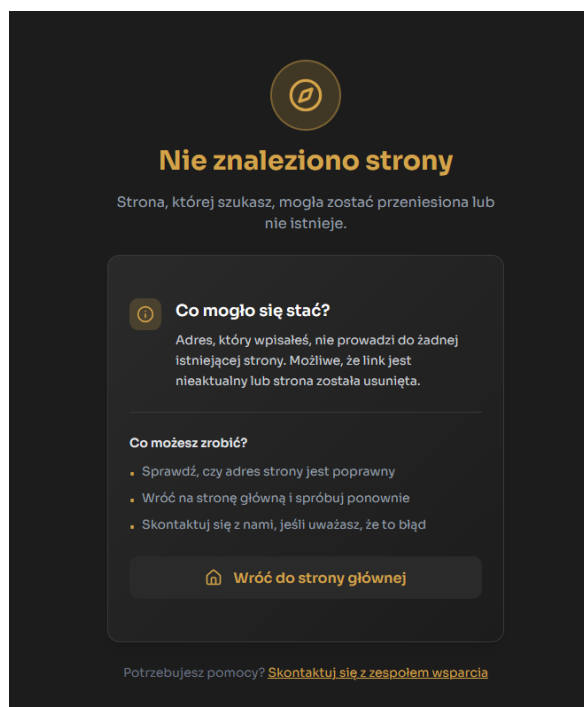
Rysunek 31: Interfejs blokady dostępu dla niezalogowanych użytkowników.

Nieodpowiednia rola (Unauthorized): Przy próbie wejścia na ścieżkę `/admin` przez użytkownika nieposiadającego roli, router blokuje renderowanie panelu. Użytkownikowi prezentowany jest komunikat o braku wystarczających uprawnień (Rysunek 32), co zapobiega wyciekowi informacji o strukturze administracyjnej systemu.

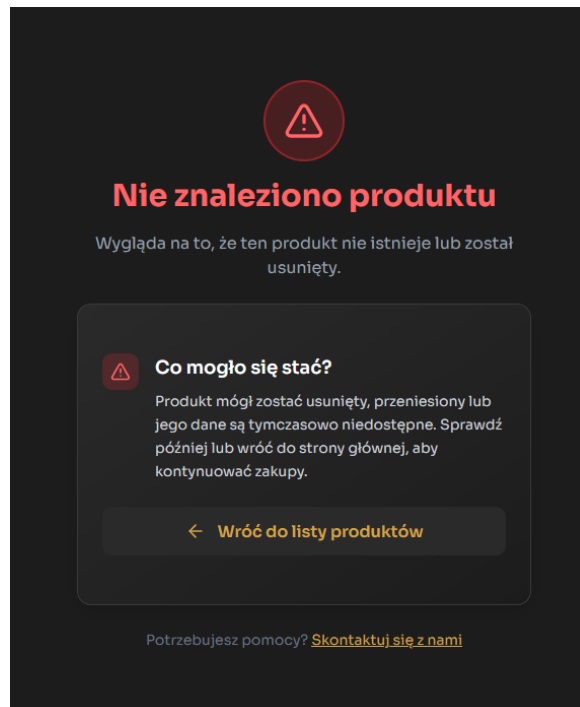


Rysunek 32: Interfejs blokady dostępu dla nieuprawnionych użytkowników.

Błąd nieznalezonego zasobu (Not Found): W sytuacjach, gdy użytkownik odwołuje się do nieistniejącego identyfikatora produktu lub błędnego adresu URL, system generuje błąd 404 (Rysunek 33). Ekran ten zawiera przycisk powrotu do strony głównej, zapobiegając utknięciu użytkownika w „martwym punkcie” aplikacji.



a) Nieznaleziona strona (URL)



b) Nieznaleziony produkt

Rysunek 33: Widoki błędu 404 w systemie KeyForge: obsługa błędnego adresu URL oraz nieistniejącego identyfikatora zasobu.

Podsumowanie

System **KeyForge** to nowoczesne rozwiązanie w obszarze e-commerce, łączące architekturę mikroservisową z wysoką dbałością o doświadczenia użytkownika. Dzięki separacji logiki biznesowej od warstwy prezentacji oraz konteneryzacji całego ekosystemu, platforma charakteryzuje się wysoką skalowalnością i łatwością wdrożenia.

Wnioski dotyczące architektury Backendowej

Warstwa serwerowa systemu KeyForge, oparta na architekturze mikroservisowej i technologii **Spring Boot**, została zaprojektowana z myślą o maksymalnej separacji odpowiedzialności oraz niezależnej skalowalności poszczególnych modułów.

- **Różnorodność warstwy danych:** Dobór technologii do charakteru danych. Relacyjne bazy **PostgreSQL** zapewniają spójność transakcyjną zamówień i produktów, natomiast dokumentowe bazy **MongoDB** oferują elastyczność niezbędną dla profilu użytkownika i dynamicznego systemu opinii.
- **Bezpieczeństwo i Tożsamość:** Centralizacja autoryzacji w usłudze **Keycloak** pozwoliła na implementację bezpiecznego standardu **OAuth2/OpenID Connect**. Dzięki temu mikroservisy są odciążone od zarządzania danymi wrażliwymi (hasła), operując jedynie na weryfikacji tokenów JWT.
- **Ekosystem AI:** Wydzielenie dedykowanego **ai-service** umożliwiło asynchroniczne przetwarzanie treści bez blokowania głównych procesów biznesowych. Integracja ta otwiera drogę do dalszego rozwoju systemu w stronę personalizowanych rekomendacji i automatycznego wsparcia klienta.
- **Niezawodność i Konteneryzacja:** Pełna konteneryzacja w środowisku **Docker** gwarantuje powtarzalność środowiska wdrożeniowego. Wykorzystanie **API Gateway** jako centralnego punktu wejścia pozwoliło na skuteczne ukrycie wewnętrznej topologii sieci i zabezpieczenie punktów końcowych mikroservisów.
- **Integracje Zewnętrzne:** Backend wykazuje wysoką dojrzałość integracyjną, co potwierdza sprawna komunikacja z systemem płatności **Stripe API** oraz obsługa webhooków zapewniająca natychmiastową realizację zamówień po potwierdzeniu transakcji.

Wnioski dotyczące Interfejsu Użytkownika

Warstwa wizualna systemu, zrealizowana w bibliotece **React.js**, została zaprojektowana w sposób spójny i intuicyjny. Kluczowe aspekty interfejsu, które wyróżniają projekt, to:

- **Responsywność i Estetyka:** Zastosowanie ciemnej kolorystyki (*Dark Mode*) oraz responsywnego designu sprawia, że system prezentuje się nowocześnie i jest w pełni funkcjonalny na urządzeniach o różnych rozdzielczościach ekranu.
- **Innowacyjność poprzez AI:** Integracja z mikroservisem AI w procesie moderacji i generowania streszczeń opinii podnosi wartość merytoryczną platformy dla klienta końcowego.
- **Grywalizacja:** Wprowadzenie systemu **KeyPoints** bezpośrednio w interfejsie koszyka i profilu użytkownika buduje lojalność klientów i zachęca do aktywnego korzystania z serwisu.
- **Bezpieczeństwo i Przejrzystość:** Wykorzystanie Keycloak oraz TanStack Router do zarządzania uprawnieniami gwarantuje, że dane administracyjne są chronione, a użytkownicy informowani o braku dostępu w sposób czytelny.